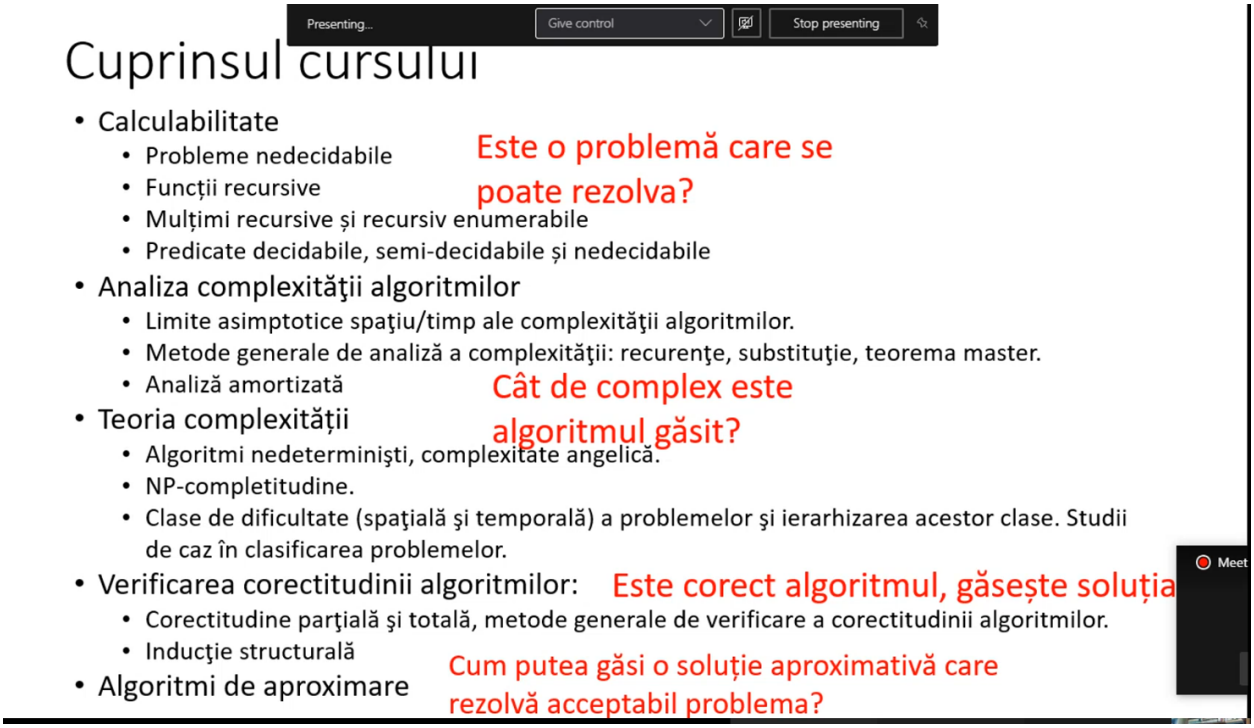


Cuprins

- Modele ale calculabilității
- Cardinalul mulțimii programelor
- Programul universal
- Teorema opririi a lui Turing
- Funcții parțial și total recursive
- Teza Church-Turing
- Mulțimi recursive și recursiv-enumerabile
- Predicate decidabile, semi-decidabile și nedecidabile
- Probleme decidabile, semi-decidabile și nedecidabile
- Teorema Rice

Radix sort – cel mai rapid algoritm de cautare; ocupa mult spatiu

Calculabilitate

Ce probleme pot fi rezolvate?

Elemente de matematica a multimilor infinite

Cardinalul multimilor infinite Cantor

Multi numarabile : multimea numerelor naturale, intregi

Corespondenta biunivoca

Multimi de puterea continuumului: multimea numerelor reale

Digonalizarea lui Cantor: : daca este o multime numarabila atunci o putem pune intr-o lista toate numele zecimale: 0, q11, q12 …… 0. q21, q 11

Elementele uei multimei numarabile se pot pune intr-o lista

Modele ale calculului

Masina Turing

Calculul lambda: LIPS

Algoritmii Markov: CLIPS, JESS, XSLT

Sistemele Post: gamatici

Teoria functiilor primitive recursive: matematica pura

Se poate demonstra ca sunt echivalente.

Masina Turing: Este o banda infinita, pe ele sunt cararctere si unitatea de comanda are stari:q1, q1. Ea citeste caracter cu caracter si in functie de starea curente or trece in alta stare or ramane in aceeasi stare si scrie sau nu un caracter be banda si ea se deplaseaza (sau nu). Este un graf de tranzitii a starilor.

Functii aritmetice f: N -> N

Calculatorum ideal, program

Numere naturale

Pentru a rezolva o problema trebuie:

1. Sa gasim o idee de abstractizare/modelare a problemei
2. Este o problema care se poate rezolva?
3. Sa gasim un algoritm de rezolvare
4. Algoritmul:
    - Sa se termine intotdeauna, pentru orice date
    - Sa resolve corect problema
    - Sa fie cat mai eficienti din punct de vedere al:
        - Timpului de executie
        - Spatiului ocupat

Algoritmii trebuie sa fie eficienti.

Algoritmil lui Kosaraju:

Complexitate O(n+m)

Sunt necesare:

*Gasirea de idei de abstractizare/ modelare

*Gasirea de algoritme

*Refolosire

*Creativitate

Algoritmul lui Kosaraju pt determinarea componentelor tare conexe: CTC(G)

Parc – ad(G)

Transpunem G

Parc – ad(GT) in ordine descrescatoare a timpurilor de finis de la primul parc in adancime

End

Componentele conexe sunt reprezentate de padurea de arbori generati

---

Pij = multimea programelor cu i intrari si j iesiri (Programele au un set finit de caractere)

Teorema: Multimea tuturor programelor este numarabila

1. P = multimea tuturor programelor = {Pij | i, j apartine N}
2. Oricare ar fi p apartine P, p este finit si are caractere in numar finit.
3. Ordonam programele dupa ordinea lexicografica in functie de ordinea caracterelor.

4. Daca le-am ordonat, atunci le putem pune intr-o lista: p1, p2, …, pN .
5. Le-am pus intr-o lista, deci multimea programelor este numarabila.

Teorema: Exista un program care simuleaza orice alt program (Se numeste programul universal)

Exista un program Q apartine P21 a.i. urmatoarele sunt adevarate in acelasi timp:

1. Q(i,x) = y, daca pi(x) = y (pi reprezinta al i-lea program din lista de la teorema anterioara punctul 4)
2. Q(i,x) = nu se termina, daca pi(x) nu se termina

Demonstratie: Construim acest Q

Citeste i
Citeste x
Reconstruieste Pi // se poate demonstra ca se poate (asta a zis proful, nimic mai mult)
y primeste Pi(x)
scrie y

Teorema opririi a lui Turing: Nu exista un program care sa poata spuna despre orice alt program daca se termina sau nu

Nu exista R apartine P21 a.i. urmatoarele sunt adevarate in acelasi timp:

1. R(i,x) = 0, daca Pi(x) nu se termina
2. R(i,x) = 1, daca Pi(x) se termina

Multimea tuturor functiilor (HOM(A,B)) e de puterea continuumului (nu e numarabila)

HOM(A,B) = {f: A -> B} nu este numarabila

Problema opririi: testare a opririi nerezolvabila?

Demonstratie teorema opririi a lui Turing:

Presupunem ca exista R apartine lui P21 a.i. putem construe programul S apartine lui P11 astfel:

S(n) = 0, daca Pn(n) nu se termina

S(n) nu se termina, daca Pn(n) se termina

S:
Citeste n
m <- R(n,n)
daca m==1 atunci while(1){}
scrie m

S(n) = Pk
Exsita k apartine lui N a.i. S este al k-lea program din lista programelor

S(k) = Pk(k) = 0, daca Pk(k) = 1
S(k) = Pk(k) = Nu se termina, daca Pk(k) se termina

Adica

S(k) = Pk(k) se termina daca nu se termina
S(k) = Pk(k) nu se termina daca se termina

Contradictie

Exemplu de program care bucleaza la infinit

i = 1

cat timp i < 0 repeta

i = i + 1

[]

tipareste i

Teorema de incompletitudine a lui Godel

Orice sistem formal care include aritmetica contine cel putin o teorema care nu se poate demonstra

(sistem formal – axiome + reguli de derivare) => Teoreme

Demonstratie

1. Godelizare – fiecarei formule i se atribuie un numar natural unic
   formule = axiome + teoreme
   variabile = produse de nr prime la puteri nr prime => nr unice
2. Teorema de genul “eu mint” nu poate fi demonstrata
   Teorema cu numarul unic k este falsa

Modelul masinii vs modelul ceasului

Masina Turing – primeste o intrare (pe banda un sir de caractere), functioneaza si cand se opreste, pe banda e iesirea – este esentiala oprirea

Acesta a fost Modelul „clasic al calcului”
Ceasul – idea nu se opreste niciodata si datele utile nu sunt cand se opreste ci in fiecare moment, este esential sa nu se opreasca

ex: sistem de operare, server

---

program – algoritm

functie

multime

predicat

Functii recursive:

- Definitie – O functie aritmetica (f: N -> N) partiala (o functie definita pe o submultime a domeniului de definitie) este recursiva daca este calculabila:
  Exista p apartine P11 a.i. f(n) = p(n), daca n apartine domeniului de definitie a lui f (def(f))
  nu se termina, altfel
- Teorema: Exista functii artimetice nerecursive (si acestea sunt mai multe).
  Demonstratie: Functiile sunt calculate de programe.
  Pentru orice functie luam programul cu indicele minim care o calculeaza.
  Multimea tuturor functiilor este HOM(N, N) – are cardinal egal cu cardinalul lui R, dar programele au cardinal egal cu cardinalul lui N => sunt multe functii necalculabile.

Pentru ca sunt mul mai multe functii decat programe, atunci sunt mult mai multe necalculabile decat calculabile! (functie calculabila = functie recursiva)
HOM(N,N) - omomorfica
Recursive AA diferit de functiile recursive de la PC. Aici functie recursiva inseamna calculabila.

Exista o alta teorie a calculului care considera:

- functii primitiv recursive
- functii recursive (adaugand operatorul de minimizare = miu(litera greceasca)); functiile recursive sunt functii recursive partial si functii recursive total

---

Ex: f(n) = 10 – n => def(f) = {9, 8, 7, 6, 5, 4, 3, 2, 1} - > partiala; recursiva

---

Exista functii totale nerecursive

Fie f aritmetica (f:N -> N) totala

f(n) = 1, daca pn(k) este definita pentru oricare k din N

f(n) = 0, altfel

f nu este recursiva

Demonstratie bazata pe reducere la absurd si principiul diagonalizarii a lui Cantor

Presupunem ca f este recursiva si totala => exista Q care o calculeaza. Atunci construim

Q(n) = Pn(n) + 1 daca f(n) = 1

Q(n) = 1, daca f(n) = 0

Dar exista k astfel incat Q = Pk => Pk(k) = Pk(k) + 1 contradictie

---

Teza lui Church – Turing

Toate functiile calculabile algoritmic sunt recursive

Multimi recursive

Definitie: O multime A inclusa in multimea numerelor naturale este recursiva daca:

1. functia ei caracteristica este recursiva
   f(n) = 1 daca n din A si 0 altfel
2. sau exista un program cu o intrare care tipareste in timp finit “Da” daca intrarea este in multime si “Nu” daca nu este in multime

Exista P astfel incat P(n) – da daca n din A, nu altfel

Multimea recursiv-enumerabile

Definitie: O multime A inclusa in multimea numerelor naturale este recursiv-enumerabila daca:

1. functia ei caracteristica este recursiva pe domeniul de definitie a lui f
   A = def (f)
2. sau exista o functie recursiva astfel incat A = def f
3. sau exista un program cu zero intrari care tipareste succesiv toate elementele multimii A
   Atentie! A este infinita

Multimi recursive: Definitie: O multime A inclusa in multimea numerelor naturale este recursiva daca:

1. functia ei caracteristica este recursiva
   f(n) = 1, daca n apartine A
   f(n) = 0, daca n nu apartine A

2. SAU exista un program cu o intrare care tipareste in timp finit „DA" daca intrarea este in multime si „NU" daca intrarea nu este in multime
   Exista p a.i. p(n) intoarce DA daca n apartine lui A
   p(n) intoarce NU daca n nu apartine lui A

Multimi recursic-enumerabile

Definitie: O multime A inclusa in multimea numerelor naturale este recursiv-enumerabila daca:

1. functia ei caracteristica este recursiva pe domeniul de definitie a lui f
   A = def(f)
2. SAU exista o functie recursiva a.i. A = def(f)
3. SAU exista un program cu zero intrari care tipareste succesiv toate elementele multimii A
   Exista p a.i. p returneaza {e1, e2, ...}. Atentie A este infinita.
   De exemplu, demonstrarea ca o formula este teorema in logica predicatelor. De exemplu, in planificarea actiunilor unui robot.

Demonstrarea teoremelor calculului cu predicat

Teoreme:

1. A recursiva => A recursiv-enumerabila
   Dem:
   A recursiva => exista p apartine P11, a.i. p intoarce DA sau NU daca intrarea este sau nu in A
   Scriem urmatorul program:
   Citeste n
   m primeste p(n)
   daca m == NU atunci while(1) {}
   scrie m
2. A recursiv-enumerabila si N-A (complementarul lui A) recursiv-enumerabila => A recurisiva (Daca o multime si complementara ei sunt recursiv-enumerabile atunci multimea initiala este recursiva.)
   Dem:
   Fie p(A)[i] este programul care genereaza toate elementele lui A la momentul i
   Analog p(N-A)[i]

   Fie programul:
   Citeste n

```
gasit primeste fals
while gasit == fals {
        daca n == p(A)[i] atunci gasit primeste DA
        daca n == P(N-A)[i] atunci gasit primeste NU
        i primeste i + 1
}
Scrie gasit
```

3. A) exista multimi recursive
   B) exista multimi recursiv-enumerabile dar nu recursive
   C) exista multimi care nu sunt nici macar recursiv-enumerabile

   Dem:
   A) Banal, multimi finite
   B) Fie A = {p apartine P11 | pm(m) se opreste} = multimea programelor care se opresc
      Trebuie demonstrat ca A este recursiv-enumerabila si ca A nu este recursiva.
      Fie programul:

```
pasi primeste 1
repeta {
        pentru i de la 1 la pasi {
                repeta {
                        daca pm(m) da un rezultat in cel mult m pasi si m nu a fost inca
                        afisat atunci {
                                scrie m
                        }
                }
        }
        pasi primeste pasi + 1
}
```

   Deci, {m | pm(m) se opreste} este recurisv-enumerabila

   Mai trebuie sa demonstram ca A nu este recursiva:

```
Presupunem ca este recursiva => f caracteristica a lui A este recursiva =>
f(n) = 1, daca pn(n) se opreste
f(n) = 0, daca pn(n) nu se opreste

putem sa construim programul: Q(n) =
        pn(n) + 1, daca f(n) = 1
        0, daca f(n) = 0
Contradictie => A recursiv-enumerabila dar nu si recursiva
```

   C) Exista functii care nu sunt nici macar recursiv-enumerabile
      Demonstratie: Fie multimea B = {m | pm(m) nu se opreste} = N-A (unde A este multimea de la punctul B), care este o multime recursiv-enumerabila dar nu si recursiva)

Daca B ar fi recursiv-enumerabila (stiind ca A este recursiv enumerabila) => A recursiva (daca o multime si complementara ei sunt recursiv-enumerabile atunci multimea in cauza este recursiva)

A recursiva => contradictie

Predicate si probleme de decizie:
Un predicat este un T pentru care T(intrare) = DA sau NU in functie de intrare (ca un if).

Un predicat T: (N x N x N x ... x N) -> {adevarat, fals} este decidabil daca:

- Multimea sa de adevar {(x1, x2, x3, ..., xn) | T(x1, x2, x3, ..., xn) = adevarat} este recursiva
- SAU exista un program p care, pentru fiecare tuplu (x1, x2, x3, ..., xn) intoarce:
  DA, daca T(x1, x2, x3, ..., xn) = adevarat
  NU, daca T(x1, x2, x3, ..., xn) = fals

Un predicat T: (N x N x N x ... x N) -> {adevarat, fals} este semidecidabil daca:

- Multimea sa de adevar {(x1, x2, x3, ..., xn) | T(x1, x2, x3, ..., xn) = adevarat} este recursiv-enumerabila si nu recursiva
- SAU exista un program p care, pentru fiecare tuplu (x1, x2, x3, ..., xn) intoarce:
  DA, daca T(x1, x2, x3, ..., xn) = adevarat
  nu se opreste, daca T(x1, x2, x3, ..., xn) = fals
- SAU exista un program p care tipareste succesiv toate tuplele (x1, x2, x3, ..., xn) pentru care T(x1, x2, ..., xn) = adevarat.

Probleme decidabile si semidecidabile si nedecidabile:

- Problema demonstrarii teoremelor in calculul cu predicate.(semidecidabila; recursiv-enumerabila).

Reducerea Turing:
Definitie – spunem ca problema A se reduce Turing la problema B daca si numai daca exista o functie F care transforma orice intrare i a problemei A intr-o intrare F(i) pentru problema B, astfel incat daca A(i) = 1 atunci B(F(i)) = 1.
Este o modalitate de a arata daca alte probleme sunt semidecidabile, nedecidabile sau decidabile daca stim deja cum sunt altele.

Teorema lui Rice:
Orice proprietate extensionala nebanala a programelor este nedecidabila.

O proprietate extensionala este o proprietate data prin enumerarea programelor care o au. De ex., proprietatea patratelor = {P1, P2, ...} unde P1, P2, ... calculeaza patrate banale (poate intr-un ciclu ceva). O proprietate extensionala nebanala este o proprietate diferita de multimea vida.

Demonstratie: presupunem ca proprietatea este decidabila si notam proprietatea cu „prop”

Fie „nonstop” un program care ruleaza la infinit.
Cazuri:

a) nonstop apartine lui prop
   exista un program Q apartine P11, Q diferit de nonstop, Q nu apartine lui prop
   putem construi programul R

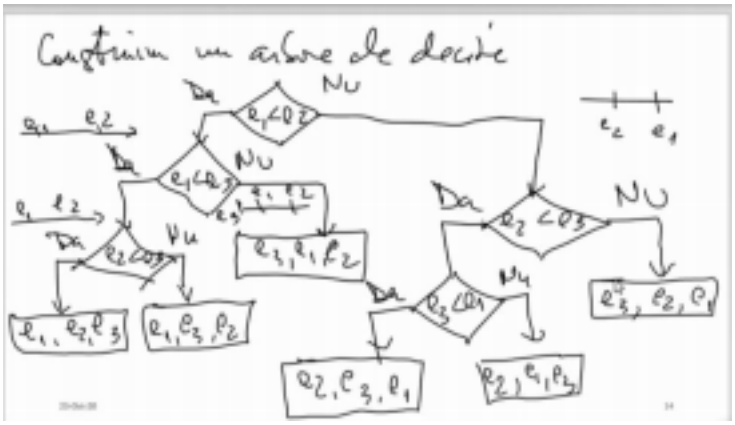
   R: {
   Citeste x
   P(x) (un program arbitrar)
   Q(n)
   }

   R = nonstop, daca P(x) nu se termina => R apartine lui prop
   R = Q(n), daca P(x) se termina => R nu apartine lui prop
   Deci, proprietatea testeaza oprirea, iar problema opririi este nedecidabila.
   Contradictie
b) nonstop nu apartine lui prop => se demonstreaza analog

Curs 4:

Analiza complexitatii algoritmilor

- Notatii de complexitate
    - Nu folosim durata in timp
    - Complexitate polinomiala (n^k) – ce ne dorim
    - Complexitate exponentiala (k^n) se numesc si “intractable”, intratabile, intractabile
- Limite asimptotice
    - **O(g(n))** = {f:N - > R+ / E c in R+, n0 in N ai.
      Oricare n >= n0, 0 <= g(n) <= c * f(n)} -> limita superioara
    - **Omega(g(n))** = {f:N - > R+ / E c in R+, n0 in N ai.
      Oricare n >= n0, g(n) >= c * f(n)} -> limita inferioara
    - **Theta(g(n))** = {f:N - > R+ / E c1, c2 in R+, n0 in N ai.
      Oricare n >= n0, c1 * f(n) >= g(n) >= c2 * f(n)} -> limita superioara si inferioara
    - **o(g(n))** = {f:N - > R+ / Oricare c in R+, E n0 in N ai
      Oricare n >= n0 avem g(n) <= c * f(n)} -> limita strict superioara
    - **w(g(n))** = {f:N - > R+ / Oricare c in R+, E n0 in N ai
      Oricare n >= n0 avem g(n) >= c * f(n)} -> limita strict inferioara
    - Astea toate sunt multimi de functii!!!
- Proprietati
    - Tranzitivitate: toate, f = O(g), g = O(h) <=> f = O(h)
    - Reflexivitate: pt astea cu litera mare
    - Simetrie: Theta, f = Theta(g) <=> g = Theta(f)
    - Antisimetrie: perechi litere mici-mari, f = O(g) <=> g = Omega(f)
      f = o(g) <=> g = w(f)
    - Proiectie: f =Theta(g) <=> f = O(g), f = Omega(g)
    - Limitele, sunt in seminar
    - In general, constantele se ignora (mai putin daca sunt la exponent)
    - O(f+g) = O (max(f,g))
    - t = o(g) ⇔ n -> INF LIM(t / g) = 0
    - t = w(g) ⇔ n -> INF LIM(t / g) = INF
- Calculul complexitatii
    - Exemplu: Complexitatea sortarii prin comparatii de chei este Omega(n*logn) ○
      Abstractizare: sortarea prin comparatia de chei <->operare pe unn arbore de decizie ○
      EX simplu: consideram sortarea a 3 elem (e1,e2,e3)
    - Arbore de decizie:

- 

  - Generalizare: sortarea este o cale in arbore; treb sa calculam adancimea maxima “h”, acest h imi da complexitatea
  - Sunt n! variante -> frunze
  - Sunt cai de lungimi diferite
  - Daca ar fi arbore complet, ar fi 2^h Frunze; n! < 2^h (avem cai de lung dif)
  - N! >= (n/e)^n (formula lui stirling)
  - 2^h > n! > (n/e)^n -> h > n * log(n) - n * log(e) -> h = Omega (n * log(n))
  - Exemplu: Complexitatea problemei Turnurilor din Hanoi ()
- NP completitudine (NP = P)
- Clase de complexitate
- Allgoritmi de aproximare

Curs 5:

- Calculul complexitatii
  - Bubblesort : caz favorarbil (Theta(n)), ocupa n +_nr elemente(= 1) + 1 celula de memorie (temp)
  - Radix, count sort: O(n), asta ocupa mult spatiu
  - Ne dorim sa o calculam exact, Theta sau O
  - Se poate calcula si aproximativ, o estimare, in baza unui algoritm deja scris
    Se doreste mate pe cat posibil

-Problemele pot fi

  - Nedecidabile, Semidecidabile, Decidabile
  - Decidabile ->Polinomiale, Exponentiale, NP complete

- Analiza estimativa a complexitatii (calitativa)
  - Exemplu:
  - Citiri, atribuiri, cicluri – se pune un „probe” in punctul cel mai des folosit, unde se fac cele mai multe repetitii (cum ar fi in cicluri), apoi se estimeaza complexitatea zonei respective
  - In baza intuitiei facem ceva matematic

  - Exemplu: coada de prioritati

- Se va vorbi de tipuri de date abstracte
- Operatii: insert, getmax, delmax
- Sigma algebra: struct matematica, le fol la inductie structurala
  Astea au asa:
  - Sorturi: stive Q, elemente E
  - Signaturi: operatorii folositi:
    - Constructori: nulari (newQ), recursivi (insert, delmax), nerecursivi (cun facem un arraylist in java, cu enumerare de elemente) -> astea genereaza cate o coada noua
    - De caracterizare\query: isEmpty (boolean)
    - Altii: getMax()
  - Axiome: cum se comporta tda urile astea
    - isEmpty(newq) = true
    - isEmpty(insert(e,q))= false
    - getmax(newq) = err
    - getmax(insert(e,q)) = max(e,getmax(q)) daca isEmpty(q) = false
      e, daca isEmpty(q) = true
- Implementare: (optima e cu heap) cu vector, cu lista, cu arbori de competitie
  Trebuie vazut complexitatile pt fiecare operatie

Curs 6:

- Exemplu: coada de prioritati – continuare
  - Axiome: (stil de programare functionala – nu permite efecte laterale, nu exista instr de atribuire)
    - Delmax(insert(e,q)) = daca isEmpty(q) atunci newQ
      Altfel daca e > getmax(q) atunci q
      Altfel insert(e,delmax(q))
- Implementare:
  - Cu un vector: initializam vectorul cu N elemente
    - Insert – O(1) daca vectorul nu e plin, O(n) daca vectorul e plin
    - Newq – O(1)
    - Empty – O(1)
    - Getmax – O(n)
    - Delmax – O(n)
  - Cu un vector sortat
    - Insert – O(n)
    - Newq – O(1)
    - Empty – O(1)
    - Getmax – O(1)
    - Delmax – O(1)
  - Cu o lista: avem nev de spatiu suplimentar pt un pointer
    - Insert – O(1)

- Newq – O(1)
- Empty – O(1)
- Getmax – O(n)
- Delmax – O(n)

▪ Cu o lista sortata

- Insert – O(n)
- Newq – O(1)
- Empty – O(1)
- Getmax – O(1)
- Delmax – O(1)

▪ Cu ABC echilibrati (arbori binari de cautare)

- Insert – O(log n)
- Newq – O(1)
- Empty – O(1)
- Getmax – O(log n)
- Delmax – O(log n)

▪ Cu arbore de competitie: fol prea multa memorie, au probleme la delmax

▪ Cu arbore binar esential

▪ Cu un heap (i guess)

- Insert – O(log n)
- Newq – O(1)
- Empty – O(1)
- Getmax – O(1)
- Delmax – O(log n)

o Exemple de utilizare:

▪ La SO – selectarea unui task cu prioritate max ▪
Algo. Dijkstra, Prim

Curs 7 (Metode de analiza a complexitatii algoritmilor)

Demonstratia este evidenta.

Copiatul este o hotie si o prostie!

Principul lui Peter:

1. Se plaseaza indivizi la intamplare intr o ierarhie(ca sa nu mai fie repetari ca la arborii de competitie)
2. Sunt lansate competitii intre sefi si subaltern, in urma carora sefii sunt eventual inlocuiti de cel mai puternic subaltern care ii depaseste.

Deficiente arbor de competitive:

- Se repeta cheile(si se vrea eliminarea repetarilor)

Folosim un heap pentru a forma un arbore esential complet.

Fii lui v[i] -> v[2i] si v[2i+1]. Tatal lui v[i] = v[[i/2]]

Algoritmul de construire a heap-ului(din arborele initial sa obtinem unul care respecta propr de heap)

Avem arborele complet sau esential complet. Variantele sunt:

- Propagare pana la locul lor(sistematic; de jos in sus)
- Scuturare pana ajung unde trebuie(sistematic; de sus in jos)
- Random(probabilistic; algoritm las vegas sau monte carlo)

Primele 2 sunt propuneri

Daca luam ca atare una din propuneri, trebuie facute mai multe ciclari, si vrem doar una.

Pentru a se face un singur ciclu -> luam pe rand elemente si le aducem la pozitia lor finala => nu e nevoie sa reluam ciclul.

Parcurgerea se incepe de la cel mai mic sef(primul nod de jos care are fii)(pt n se va incepe de la v[[n/2]], v de parte intreaga de n/2)

Concluzie: trebuie analizate toate variantele si gasita cea mai optima

Heap-ul binar poate fi memorat intr-un vector.

Algoritm heap(v, n; v)

Pentru i<-[n/2] la n repeta cerne(v, n, I; v);

Complexitate = O(n/2) * O(cerne) = O(n) * O(cerne) = O(nlogn) -> corect(e estimata), dar poate fi mai precis(O(n))

Algoritm cerne(v, n, I, v)

Muta = i

Repeta ctr<- muta

{

Daca 2crt <=n si v[2crt] > v[crt] // sa nu iesim din arbore

Muta = 2crt;

Daca 2crt + 1 <= n si v[2crt + 1] > v[muta] // muta e maximul curent

Muta = 2crt + 1

Daca crt != muta

Temp = v[crt]

V[crt] = v[muta];

V[muta] = temp;

} pana cand crt = muta

Curs 8(Metode de analiza a complexitatii algoritmilor; Divide et impera)

Tipuri de heap:

- Binar – am discutat anterior; poate fi memorat intr un vector(“liniarizare”)
- Fibonacci
- Binomial
- Leaftist
- …

Restul sunt generalizari

Heap ul fibonacci iese mai bine la analiza amortizata

Se descompune problema pana la subprobleme elementare.(quicksort foloseste memorie suplimentara fata de divide&impera)

Recurente: -Exprimarea tipului de executie printr-o relatie recursiva T(n) = f(T(g1(n)), T(g2(n)), T(g3(n)), T(g4(n)),.... T(gk(n))):
-T(n) = aT(n/b) + f(n) -T(n) = T(n-1) + T(n-2) -T(n) = 2T(n-1) + 1 -In scopul gasirii unei formule nerecurenta, de exemplu: T(n) = O(n*log(n))

Merge sort – sortare prin interclasare

Algoritm merge-sort(v, p, q) \\ primul apel : merge-sort(v, 1, n)

Daca p < q

Atunci r<-[(p + q) / 2]

Merge-sort(v, p, r)

Merge-sort(v, r+1, q)

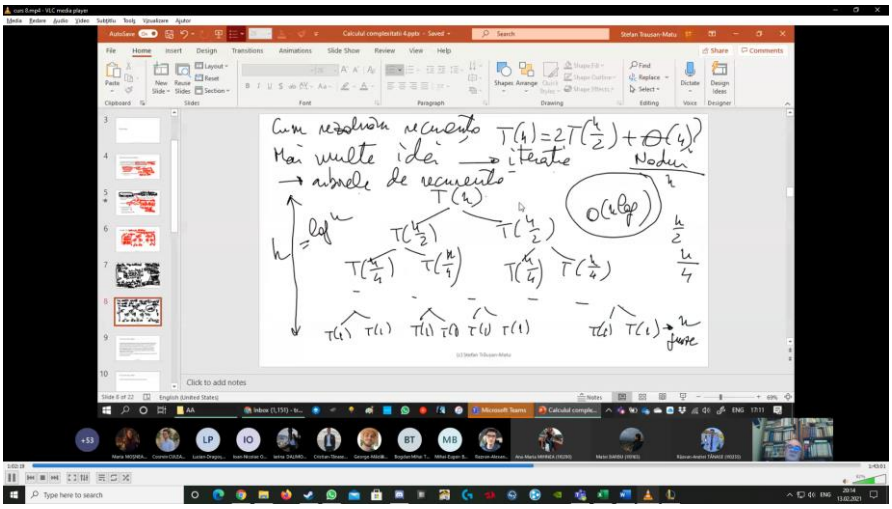
Merge(v, p, r, q)

Complexitate:

- Descompunerea : Θ(1)
- Recompunerea : Θ(n)

Complexitatea poate veni si de la prelucrari aditionale.

Putem rezolva recurenta T(n) =2 * T(n/2) + teta(n) cu un arbore de recurenta



Turnurile din Hanoi

Algoritm Hanoi(n, A, C, B)

Daca n > 2 at

Hanoi(n-1, A,B,C)

Mutadisc(A,C)

Hanoi(n-1, B,C,A)

Complexitatea: T(n) = 2T(n-1) + 1 => $O(2^n)$

Exemplu recurenta: T(n) = 2T(n-1) + 2^0 2T(n-1) = 2*2T(n-1) + 2^1 2^2T(n-2) = 2^3T(n-3) + 2^2 2^3T(n-3) = 2^4T(n-4) + 2^3 ---------- 2^qT(n-q) = 2^(q+1)T(n-(q+1)) + 2^q Adunam ecuatiile: => T(n) = 2^(q+1)T(n-(q-1)) + 2^0 + 2^1 + .. + 2^q => T(n) = O(2^n) (complexitatea este exponentiala)

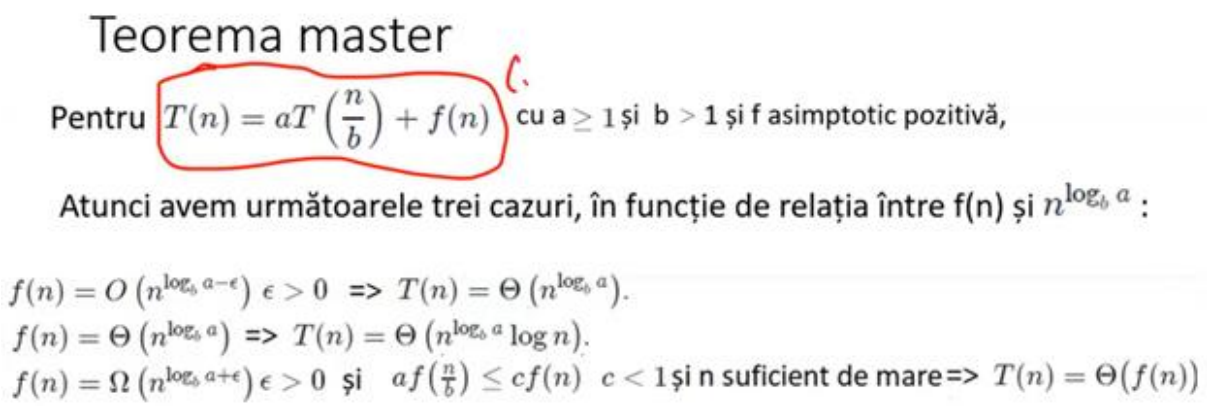
Metode rezolvarea recurentelor: -Iteratie - ca la Turnurile din Hanoi -Substitutie - intuitia solutiei si demonstratie prin inductie -Arbori de recurenta -Teorema Master

Curs 9(Metode de analiza a complexitatii algoritmilor)

Th Master:

Pt T(n) = a * T(n/b) + f(n) cu a>=1 b>=1

1. F(n) = O(n^(logb(a-e))), e>0 => T(n) = Θ (n^logb(a)) -> I domina
2. F(n) = Θ (n^logb(a)) => T(n) = Θ ((n^(logb(a))) * log n) -> I si II sunt la fel
3. F(n) = Ω($n^{\log_b a+e}$) e > 0 si a*f(n/b) <= c * f(n), c < 1 si n suficient de mare => T(n) = Θ (f(n)) -> II domina



Analiza amortizata

- Secventa de operatii asupra unei structure de date
- Costul unei operatii costisitoare se poate amortiza astfel incat el devine mic in ansamblu
- Difera de analiza pe cazul mediu prin faptul ca nu depinde de probabilitati ci este o complexitate medie in cazul cel mai defavorabil.

Metode de analiza amortizata:

- Metoda agregarii
    - Se ia costul maxim al operatiei cele mai costisitoare, efectuate de n ori
    - Am calculate un cost amortizat pe toata structura de date – ar fi mai bine sa calculam pe fiecare operatie
    - Observatii critice
        - Nu se pot face n operatii de popm, fiecare cu n iteratii – ar trebui sa am inainte de fiecare popm cu n iteratii, n push-uri => trebuie gasita alte metoda
        - La metoda agregarii am calculate un cost amortizat oe tiata structura de date- ar fi mai bine sa calculam pe fiecare operatie

- Metoda creditelor(a cotarii)
    - Cotarea fiecarei operatii in parte – cost amortizat pe operatie
    - punem atat costul push-ului, dar si al unui viitor pop. Cand facem pop platim aceasta operatie prin creditul pus la push
    - pt n operatii costul va fi O(1), fata de O(n) cum este la metoda agregarii

- Metoda potentialului
    - -> Complexitatea amortizata va fi cost real + diferenta de potential
    - -> La functia pop_m functia de potential este:
        - $\hat{C}push = Cpush + (\Phi i - \Phi(i-1))$
        - $\hat{C}pop\ i = Cpop\ I + (\Phi i - \Phi(i-1)) = 1 + (-1) = 0$
        - $\hat{C}pop\ m = Cpop\ m + (\Phi i - \Phi(i-1)) = \min\{k,s\}$

Functie de potential care reflecta “esenta” structurii de date si care se foloseste pentru a calcula un cost amortizat pe operatie

1.$a^{\log_b n} = n^{\log_b a}$

2.$O(g(n) + h(n)) = O(\max\{g(n), h(n)\})$

3.Analiza amortizata nu depinde de probabilitati (nu este un calcul pe medie, cum ar fi la arborii optimi la cautare)

4.Stiva cu pop multiplu Algoritm pop (s,k;s) i <- k cat timp i>0 si not_empty (S) repeat pop(s;s)

Curs 10

NP- Completitudine

Sunt 2 tipuri de probleme:

Tractabile , care au o complexitate polinomiala

Intractabile, care au o complexitatea exponențială (Turnurile din Hanoi)

Intre aceste 2 clase, exista o clasa de **probleme NP – complete** si nu stie daca ele sun tractabile sau intractabile. Pentru ele se stie ca exista algoritmi intractabili, se pot rezlva cu complexitate exponentiala, **dar nu exista, nu se stie, dar nici nu s-a demonstrat ca nu exista algoritmi polinomiali** . In general, sunt intractabile, in anumite cazuri putem gasi rezolvari polinomiale, **gen turnurile din hanoi cu n=3.** Complexitatea in mod normal la turnurile Hanoi este de 2^n-1.

In acest capitol, ne referim doar la probleme de decizie. Probleme care intorc DA/NU, TRUE/FALSE, SUCCES/FAIL, avand o intrare. Exista si **probleme de optim si probleme de constructie.**

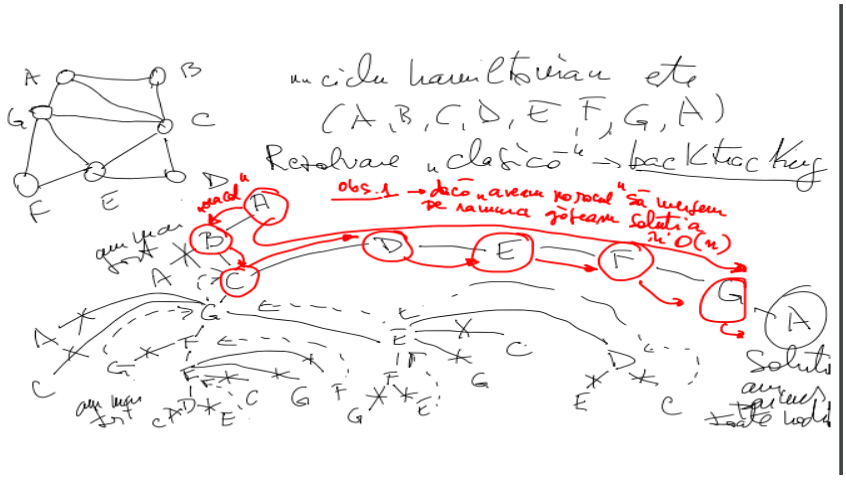
Problema ciclului hamiltonian:

• Pentru un graf neordonat G=(N,A) să se decidă dacă există un drum care trece prin toate nodurile o singură dată și se întoarce în nodul de plecare (e ciclu) – problemă de decizie

• (observație 1) problemă de optim – problema comis-voiajorului – există costuri pe arce și se cere să se găsească ciclul de cost minim

• (observație 2) problemă înrudită – ciclu eulerian (Euler) – problema podurilor din Konigsberg – în loc de noduri – arce

Exemplu la ciclul HAMILTONIAN:



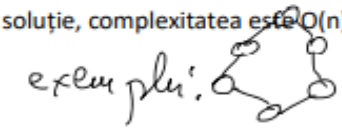
Rezolvarea clasica - > BackTracking (Ca un fel parcurgere a arborelui in adancime) Stelutele in desenul lui Trausan inseamna ca in nodul cela am mai fost si nu putem merge acolo (gen, e linie taiata, nu steluta). Se implementeaza cu apeluri recursive.

Observații :

Observații

$G = (N, A)$, $A \subseteq N \times N$

1. Dacă „avem norocul" să mergem pe ramura din arbore care duce la soluție, complexitatea este O(n) unde n este numărul de noduri
2. Pentru grafuri particulare se poate găsi soluția în O(n)



3. Dacă se dă o secvență de noduri se poate verifica în timp polinomial dacă este soluție:
   de exemplu: <A,B,C,D,E,F,G,A> e soluție, dar următoarele nu sunt:
   <A,B,C,G,D,F,G,A>
   <A,B,C,F,G,A>
4. Dacă aș avea un „oracol", care să îmi spună la fiecare decizie pe cale să o iau, complexitatea ar fi polinomială
5. Dacă aș face în paralel explorarea (dar fără overhead, ideal), complexitatea ar fi polinomială

Algoritm de verificare

Algoritmul primeste G (N, A si secventa) -> DA/NU

1. Daca cardinal (secevnta) != cardinal (N+1) - > intoarce NU // sunt mai putine noduri | **O(1)**
2. Verficam ca in secventa nu se repeta nodul cu exceptia extremitatilor (primul si ultimul) | **O(n)**
3. Facem un ciclu in care verificam ca oricare (Xi, Xi+1) din secventa, unde i=1:n-1 (Xi, Xi+1) € A si (X1 (primul nod) = Xn+1 (ultimul nod) | **cardinal (A) = O(n^2)**

Complexitatea: O(n^2) - > polinomial

Algoritmi determiniști și nedeterminiști

• Algoritmi determiniști – cei „clasici”, „reali” - **comportarea este întotdeauna aceeași**, la orice execuție a algoritmului

• Algoritmi nedeterminiști - „abstractizare” – **comportarea este nedeterministă** – diferă de la o execuție la alta a algoritmului

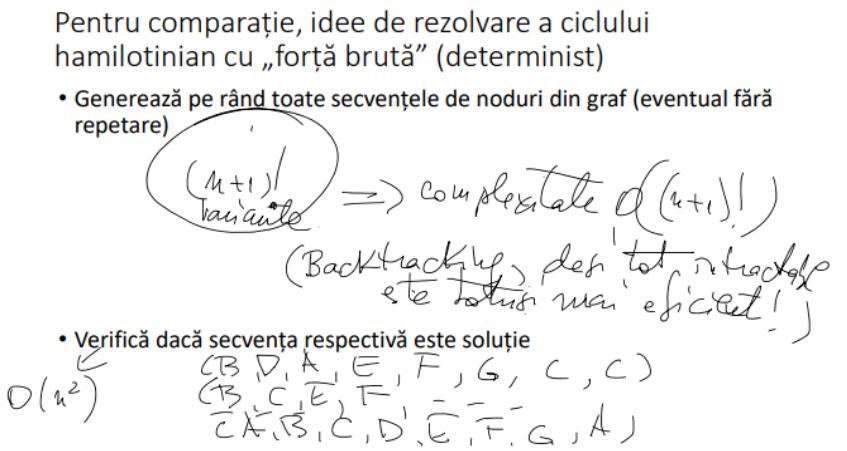
Algoritmi nedeterminiști

- Instrucțiuni specifice, suplimentare:
  - choice(A) – A este o mulțime de variante, de exemplu:
    - nod-curent ← choice(N) – în acest punct al execuției algoritmului se generează în paralel #N ramuri (numărul de elemente) de continuare a algoritmului
    - Alt exemplu x← choice(0,1) sau x←choice (x,y,z,u)
  - succes
  - fail
- Observații
- Sunt algoritmi de decizie (succes → „da”; fail → „nu”)
- Execuția algoritmului se termină la primul „succes” sau, dacă nu există succes (răspunsul e „nu”) când se termină toate ramurile cu „fail”

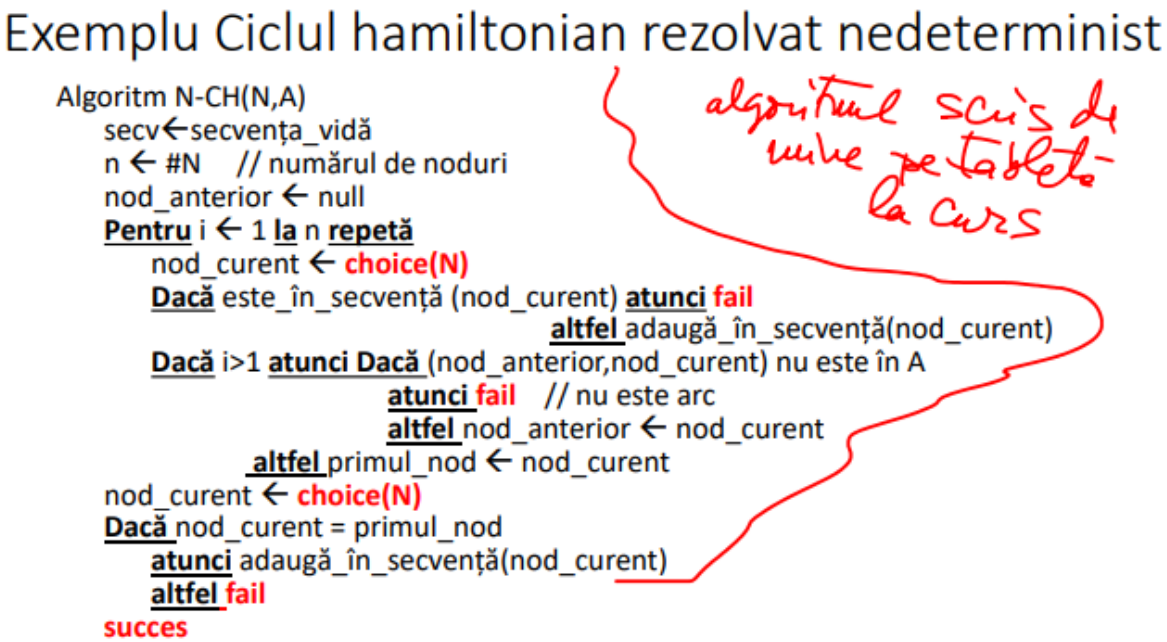
Tot ce e mai sus e abstract, un model matematic.. (Nu o sa rulezi un algoritm de genul) Foloseste la demonstratii matematice , folosind aceasta abstractizare (O lume ideala), cum nu exista oracol.

Rezolvarea clasică a ciclului Hamiltonian cu brute force

Se incearca toate variantele cu brute force | Complexitatea O(n+1)! (factorial)



Exemplu Ciclul hamiltonian rezolvat nedeterminist Idee similara cu cea brute force (de mai sus), dar nederminista.



Curs 11

Spatiul starilor

Spațiul stărilor

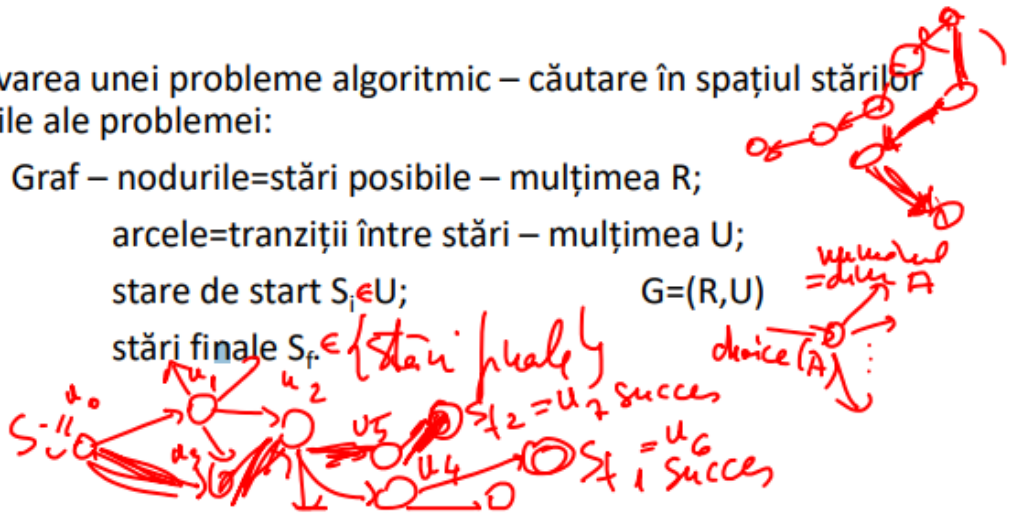
Rezolvarea unei probleme algoritmic – căutare în spațiul stărilor posibile ale problemei:

Graf – nodurile=stări posibile – mulțimea R;

arcele=tranziții între stări – mulțimea U;

stare de start $S_i \in U$; G=(R,U)

stări finale $S_f \in$ {stări finale}

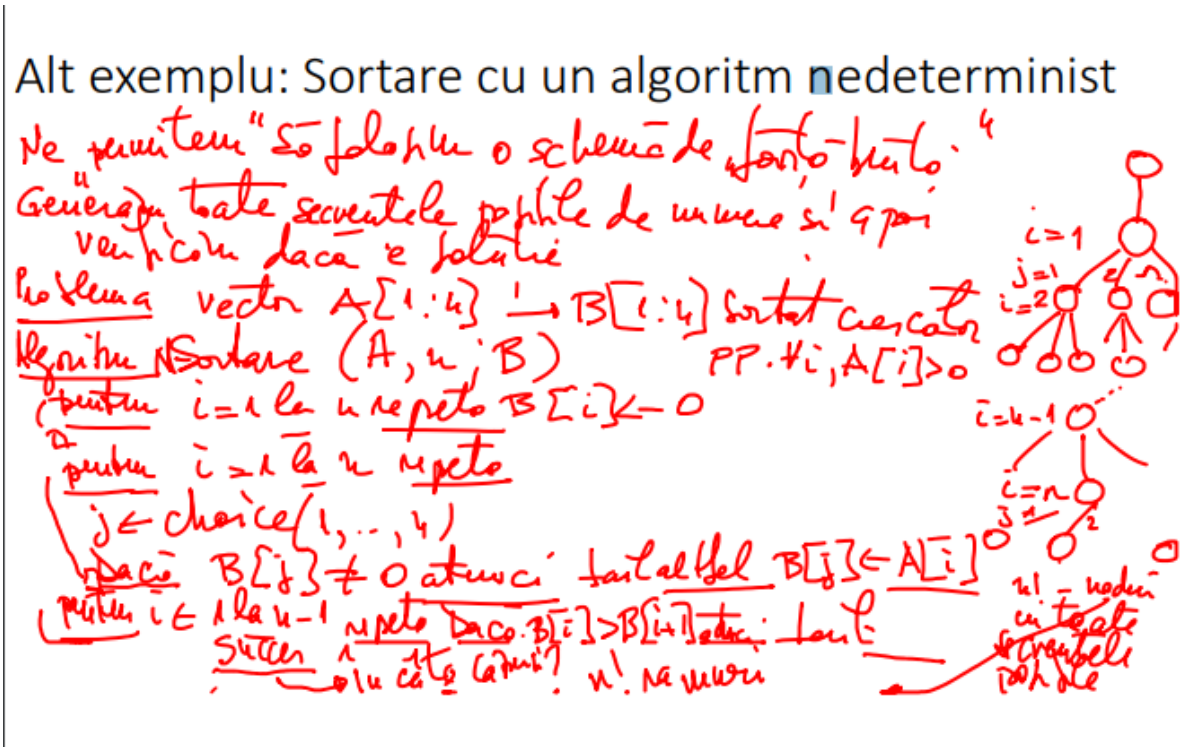


Parcurgerea spațiului stărilor

- Algoritmi determiniști („clasici”, „efectivi”) – se face doar pe o cale
- Algoritimi nedeterminiști (abstractizare) – se poate face o parcurgere în „paralel”, cu ramificații la stările corespunzătoare instrucțiunilor „choice”

Alt exemplu: Sortare cu un algoritm nedeterminist

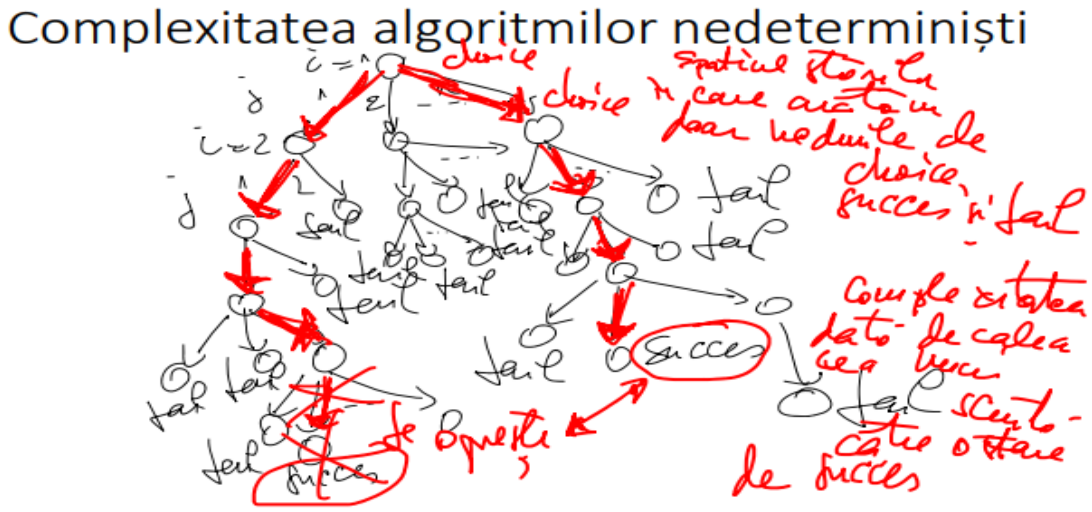
- Ne permitem sa folosim o schema de forta bruta
- Generam toate secventele posibile de numere
- Verificam apoi daca e solutie
- Problema : avem un vector A[1:n] -> B[1:n] vrem ca B sa fie sortat crescator



Complexitatea angelică (este abstractă)

Complexitatea unui algoritm nedeterminist este dată de complexitatea căii celei mai scurte către o stare de succes sau, dacă problema nu are soluție (întoarce „fail”- echivalentul lui „nu”, „fals”), complexitatea celei mai lungi căi către o satre „fail”.

Complexitatea angelica => fie **cea mai scurta cale catre SUCCES (daca are solutie)** fie cea mai lunga **cale catre un FAIL (daca nu are solutie)**



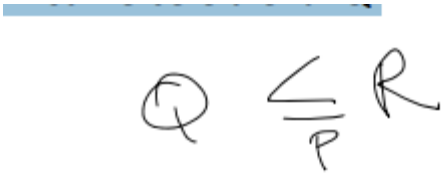
Clasa problemelor P (PTIME) și clasa problemelor NP (NPTIME)

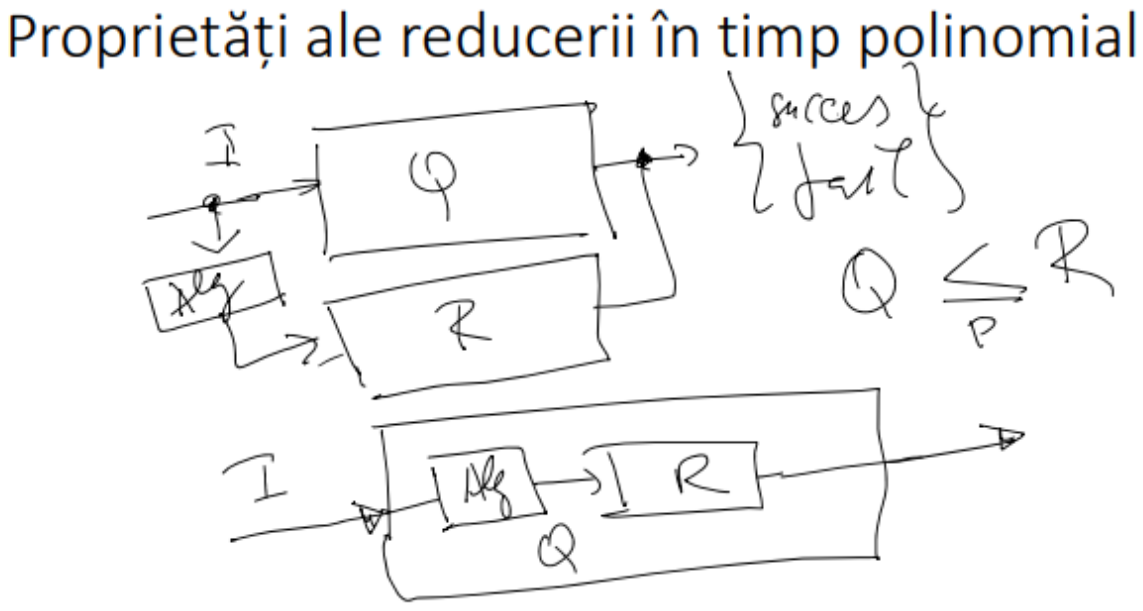
• **Probleme P (PTIME)** – probleme care se pot rezolva cu o complexitate temporală în timp polinomial cu un algoritm determinist

• **Probleme NP (NPTIME)** – probleme care se pot rezolva cu o complexitate (angelică) temporală în timp polinomial cu un algoritm nedeterminist

Reducere în timp polynomial

• O problemă Q se reduce în timp polinomial la problema R dacă și numai dacă există un algoritm determinist (Alg) cu complexitate polinomială, care transformă intrările lui Q în intrările lui R astfel încât ieșirile lui R să fie cele ale lui Q





Probleme:

• NP-Dificile (NP-Dure, NP-Hard)

Probleme la care se pot reduce în timp polinomial toate problemele din NP

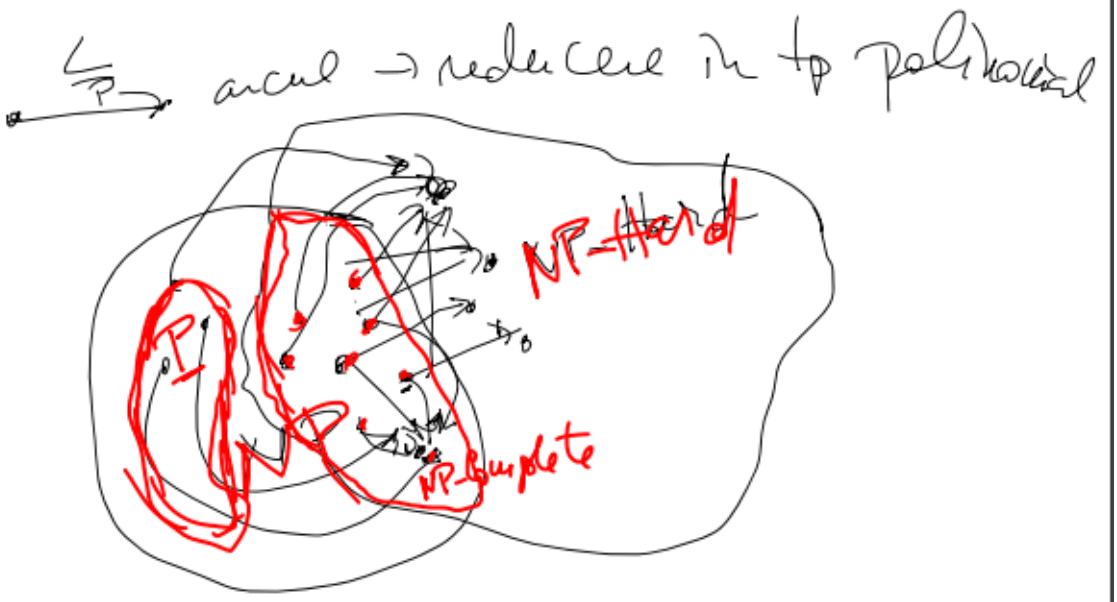
• NP-Complete

NP-Dificile, care aparțin lui NP

Altfel spus (alternativ):

Probleme la care se pot reduce în timp polinomial toate problemele din NP

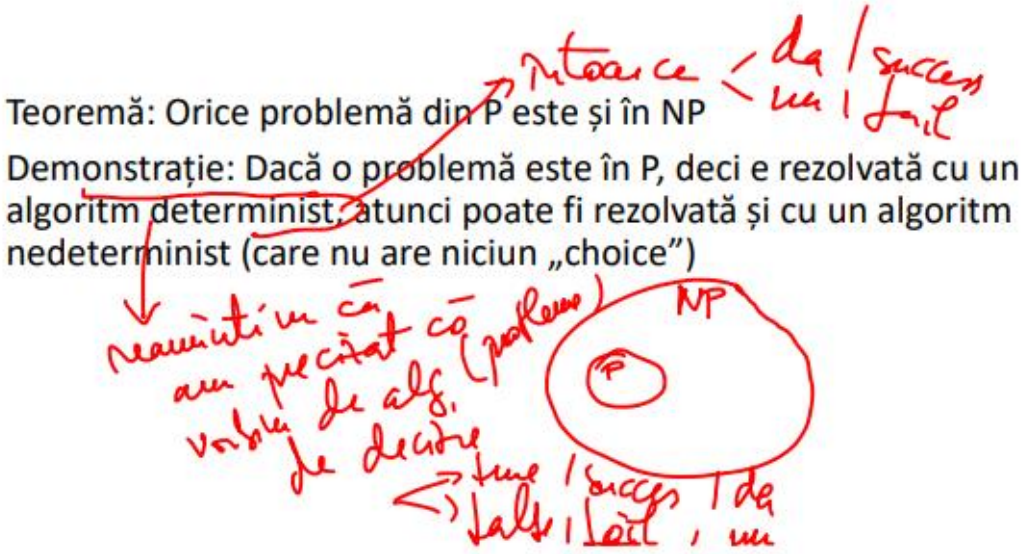
Probleme din NP



Multimea probemelor NP HARD care sunt in NP se **numeste multimea NP Complete**

Teoremă: Orice problemă din P este și în NP

Demonstrație: Dacă o problemă este în P, deci e rezolvată cu un algoritm determinist, atunci poate fi rezolvată și cu un algoritm nedeterminist care nu are niciun „choice”

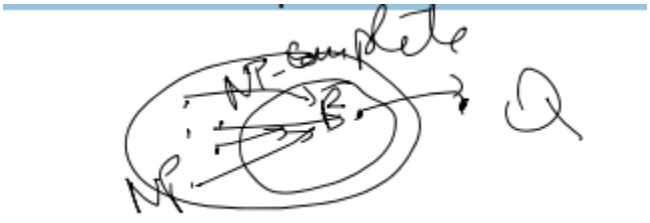


P si NP complete sunt disjuncte?

Exista probleme in NP complete care sunt si in P?

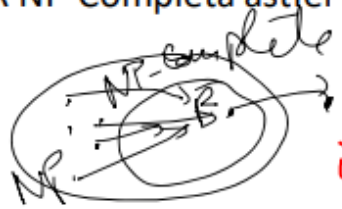
Daca ar fi astfel de probleme, ar rezulta ca P = NP

Teoremă: O problemă Q este NP-Dificilă dacă și numai dacă există o problemă R NP-Completă astfel încât R să se reducă polinomial la Q



Teoremă: Problemele NP-Complete formează o clasă de echivalență în raport cu reducerea polinomială

- Teoremă: O problemă Q este NP-Dificilă dacă și numai dacă există o problemă R NP-Completă astfel încât R să se reducă polinomial la Q



Dem. $R \in NP\text{-Complete}$
$\Rightarrow \forall S \in NP, S \leq_P R$
ipoteză $\Rightarrow R \leq_P Q$ $\Rightarrow$
$\Rightarrow \forall S \in NP, S \leq_P Q \Rightarrow Q \in NP\text{-hard}$
tranzitivă ≤

- Teoremă: Problemele NP-Complete formează o clasă de echivalență în raport cu reducerea polinomială

Dem. Fie două probleme Q_1 și Q_2 din NP-Complete
$\Rightarrow \forall Q' \in NP, Q' \leq_P Q_1, Q_1 \in NP\text{-Complete} \; Q_2 \leq_P Q_1$
Valabil pentru $Q_2 \Rightarrow Q_1 \leq_P Q_2 \Rightarrow Q_1$ și Q_2 sunt
aceeași clasă de echivalență

Activate Windows

Teoremă: Reducerea în timp polinomial este tranzitivă

Teoremă: Dacă o problemă NP-Completă este în P, atunci P=NP Demonstrație: Fie R atât în P cât și în NP-Complete
Pentru oricare Q din NP, Q se reduce în timp polinomial la R

Teoremă (Cook): Există o problemă NP-Completă: Problema satisifabilității formulelor calculului propozițional în forma normal conjunctivă (CNF)

Teoremă: Dacă o problemă NP-Completă este în P, atunci P=NP

Demonstrație: Fie R atât în P cât și în NP-Complete

➔ Pentru oricare Q din NP, Q se reduce în timp polinomial la R

$\forall Q \in NP, Q \leq_P R$

dar $R \in P \Rightarrow \forall Q \in NP \quad Q \leq R$ și $R \in P \Rightarrow$

$\Rightarrow$ avem un algoritm polinomial și pentru $Q \Rightarrow Q \in P \Rightarrow NP \subseteq P$

dar am arătat că $P \subseteq NP \Rightarrow$

$\Rightarrow P = NP$

Teoremă (Cook): Există o problemă NP-Completă: Problema satisifabilității formulelor calculului propoziţional în forma normal conjunctivă (CNF)

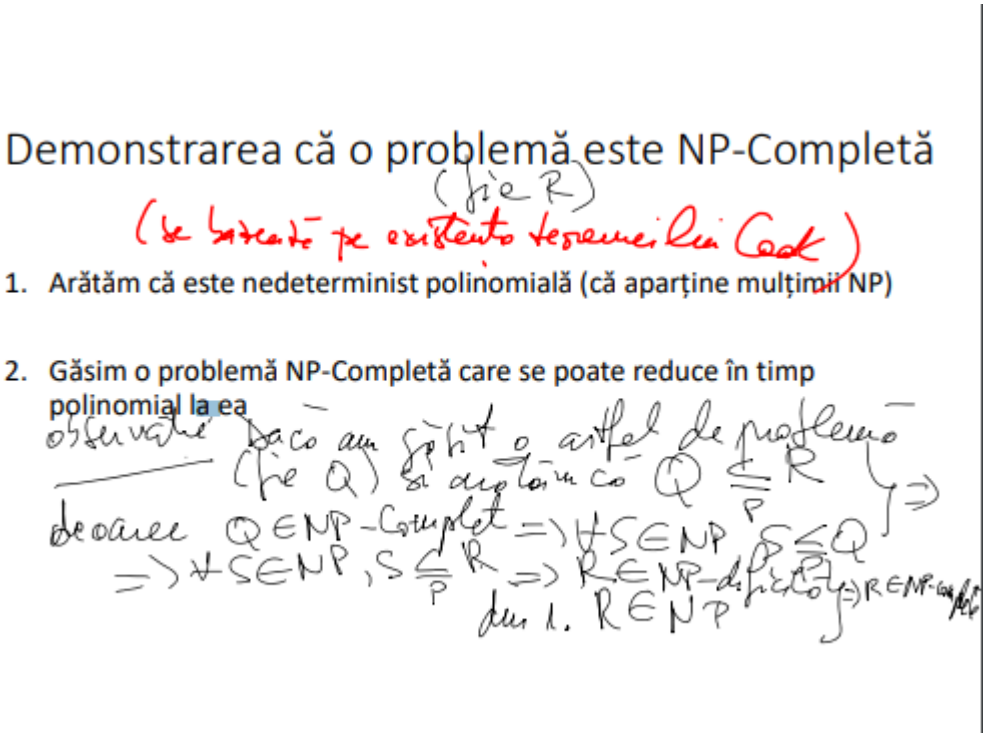
Curs 12

De ce studiem NP-Completitudinea?

De ce studiem NP-Completitudinea?

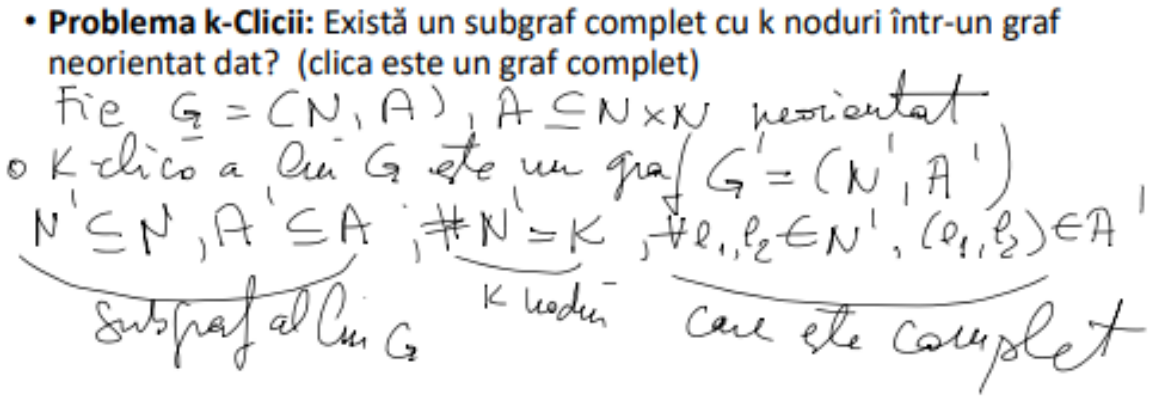
- Pentru multe probleme cu care ne confrontăm foarte des în realitate nu s-au găsit (încă?) algoritmi polinomiali
  - https://en.wikipedia.org/wiki/List_of_NP-complete_problems
- S-au găsit regularități, elemente comune între aceste probleme
  - → Modelul matematic al algoritmilor nedeterminiști

Demonstrarea că o problemă este NP-Completă



Exemplu de demonstrare că o problemă este NP-Completă – Problema K-Clicii

• Problema k-Clicii: Există un subgraf complet cu k noduri într-un graf neorientat dat? (clica este un graf complet)



NP-Completitudine.

Demonstratie ca o problema R este NP-Completa.

O problema este NP-Completa daca :

- Aratam ca este nedeterminist polinomiala adica este in NP $\Leftrightarrow$ R $\in$ NP
- Gasim o problema NP-completa care se poate reduce in timp polynomial la ea. (daca toate problemele din NP se reduc la ea)

Q $\leq_p$ R (daca la Q se reduc in timp polinomial toate problemele din NP, si aratand ca Q se reduce in timp polynomial la R, atunci clar toate se reduc la R).

Daca este o clica => nodurile clicii corespund la literali = 1 => variabilele respective 1 sau 0 (daca e x_i sau not(x_i) - negat)

Daca F satisfiabila adica F = 1 => in fiecare c_i e cel putin un literal egal cu 1.

- Luam nodurile corespunzatoare acestor literali si ele prin constructie sunt arce intre ele (intre toate).
- Obs. Sunt arce intre toate nodurile selectate pentru ca arcele sunt intre termeni diferiti si fiecare nod este in alt conjunct(pentru ca am in fiecare conjunct unul din cei cu 1)
- arcele nu pot fi intre o variabila si una negata (din constructive)

~ Găsim o problemă în NP-Complete care se reduce în timp polinomial la problema k-clicii.

- Trebuie să arătăm că trecerea de la 3CNF la k-Clică (construirea grafului) se face în timp polinomial cu un algoritm determinist

Scriem algoritmul de transformare:

N = multimea nodurilor =

A = multimea arcelor =

Algoritm.(determinist)

$F(x_1, x_2, \ldots\ldots, x_l) = C_1 \wedge C_2 \wedge \ldots\ldots \wedge C_n$

C_i =

Algoritmul construieste graf (F -input)

F= formula;

l = nr de variabile

N ←{}

A ←{}

Pentru i = 1 la l repeta

Pentru j = 1 la 3 repeta

N ← N

Pentru fiecare din N repeat

Pentru fiecare

Daca p!=i si nu avem si o variabila si ea negata

Atunci A←A

Complexitate : $O(m^2)$

- **Teoremă** - Se poate arăta că dacă o formulă este satisfiabilă, se poate obține o formă CNF a ei cu un număr polinomial de conjuncte (termeni) în funcție de numărul de variabile

In cazul 2CNF problema satisfiabilitatii este in **P.**

Exemplu formula 2-CNF :

$F(x_1, x_2, x_3) = (x_1 \ \ !x_2) \wedge (!x_1 \ \ x_3) \wedge (x_1 \ \ x_2) \wedge (!x_1 \ \ !x_2)$

$F(1, 0, 1) = 1$

Idee dem. : Cand sunt doar cate 2 literali intr o disjunctie putem construe un graf in modul urmator :

Pasul 1. – transformam fiecare termen/conjunct in doua implicatii –

Plecam de la formula B->C <=> non(B) C

$l_1 \ l_2 \ \Leftrightarrow \text{non}(l_1) \rightarrow l_2$

Pasul 2. – construiesc un graf in care nodurile sunt variabile si arcele sunt implicatii

TEOREMA. *Daca exista o cale de la o variabila la ea negata sau invers F este nesatisfiabila.*

Un 2 CNF se rezolva in timp polynomial desi in general, CNF este o problema NP Completa.

Se poate scrie un algoritm determinist care determina problema in timp polynomial.

Teoremă (Cook)

• Problema SAT este în P dacă și numai dacă P=NP

1. Problema SAT este în P -> P=NP

2. P=NP -> Problema SAT este în P

Trebuie sa dem cele 2 implicatii de mai sus.

NP-Completitudine

Toate problemele din NP se reduc la SAT.

CNF = conjunctie de termeni

Fiecare termen este o disjunctie de literali(variabile care pot fi simple sau negate).

Nr de conjuncte este polinomial.

Teoremă (Cook)

- Problema SAT este în P dacă și numai dacă P=NP

1. Problema SAT este în P → P=NP

2. P=NP → Problema SAT este în P

Pentru pasul 2 avem : P=NP → Problema SAT este în P

Demonstratie : Construim un algoritm nedeterminist pentru SAT

F (x1, x2, ... xk) = ^ ^....^, unde

Pentru i = 1 la k repeta

X_i ← choice (0, 1) // aici avem toate combinatiile, (dupa ciclul asta)

Pentru i = 1 la m repeta{

este1 ←fals

Pentru j = 1 la l_i repeta {

Daca

Atunci este1 ←adevarat

}

return fail

}

return succes

Pentru pasul 1 avem: Problema SAT este în P → P=NP , adica orice pb din NP se reduce polinomial la SAT.

Deci aratam ca:

- **Pentru oricare Algoritm D din NP construim o formula → $F_{Alg, D}$**
- **Alg apartine NP, cu complexitate polinomiala(n);**

Trebuie sa construim pentru fiecare algoritm din NP si orice date de intrare o formula CNF ⇔alg se termina cu succes formula e satisfiabila;

Variabilele algoritmului Alg

- In algoritm nu apar constante. Constantele sunt reprezentate prin variabile considerate ca

parametrii suplimentari algoritmului

- Initial valoarea oricarei variabile a algoritmului este 0. Exceptie fac doar varaibilele cu rol de

parametru, initializate cu datele D ale algoritmului sau cu valorile cosntatne reprezentate de

unele variabile (valori considerate parte a datelor D)

- Consideram ca valoarea oricarei varaibile este reprezentata prin w biti, numerotati 1…w de

la dreapta la stanga. Prin convetie o valoarea cu bitul de rang 1 egal cu 1 corespunde valorii

adevarat, iar o valoare cu bitul de rang 1 egal cu 0 corespunde valorii fals

- Variabilele algoritmului Alg sunt scalare, vectorii fiind reprezentati prin multimi de variabile

scalare

Variabilele formulei F_(alg,d)

- B(x,i,t)

X in Var, I in 1….w, t in 1…p(n)

B(x,i,t) corespunde bitului de rang i al valorii variabilei x la momentul t

B(x,i,t) = 1 arata ca bitul i este 1 la momentul t, iar B(x,i,t) = 0 arata ca bitul i este 0 la momentul t

- S(k,t)

o S(k,t) =1 indica executia instructiunii k la momentul t, iar

o S(k,t) = 0, arata ca la momentul t, nu se executa instructiunea k

Structura formule F_(alg,D) - este in format CNF

F_(alg,d) = Init ^ Start ^ Serial ^ Flow ^ Eval ^ Stop

1) **Init** = ^ $T_{x,\ i}$ → initializeaza variabilele programului i - Init = ^ T_(x,i), tot toate varaibilele x si toti bitii i=1,…w

$T_{x,i}$ = B(x, i, 1) daca x e parametru si bitul i al acestuia este 1

$T_{x,i}$ = !B(x, i, 1) daca x e parametru si bitul i al acestuia este 0 sau x este variabila interna (care au valoarea initiala 0 prin conventia stabilita);

Obs: init este in forma CNF

2) **Start** = S(1,1) ^ !S(2,1) ^ ..... ^ !S(q, 1) – adica se executa doar instructiunea 1

E in forma CNF

3) Serial = pe orice ramura executia este seriala

Serial = S(i,t)

Serial => t=1 pana la p(n) ^ S(i,t) ^ (^ !(S(i,t) ^ S(j,t))) cu i si j = 1,…q

Nu e e in forma CNF -> transformam in CNF



4) **Flow** = reprezinta fluxul instructiunilor unde avem : - succesiune simpla

-salt cond./necondit.

- succes

-fail

Depinde de starea i la momentul t : S(i,t).

Flow = S(i, t) Next(i, t)

Trebuie sa calculam Next (i, t) ce va avea mai multe variante.

- Succesiune simpla (nu este ceva ce duce la un salt)

Next(i, t) = S(i+1, t+1)

- Salt neconditionat

Next(i, t) = S(j, t+1)

- Salt conditionat

Next(i,t) = [B(x, 1, t) ^ S(j, t+1)] V [!B(x, 1, t) ^ S(i+1, t+1)]

- Succes si fail :

Next(i, t) = S(i, t+1), e CNF

5) Eval - S(i, t) => Next(i, t)

Aici avem flux de date, modificam B-ul.

Avem mai multe cazuri:

Caz1) Instructiunea i nu e o atribuire

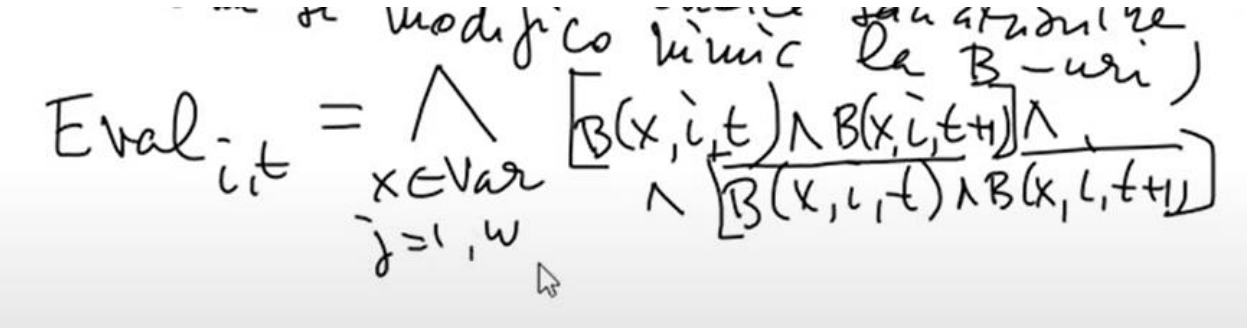
Caz 2) Choice x←choice(v1, v2, ... , vk)

Caz 3) Atribuire x←expresie

Caz 1) Instructiunea i nu este choice sau atribuire

(nu se modifica nimic la B-uri)

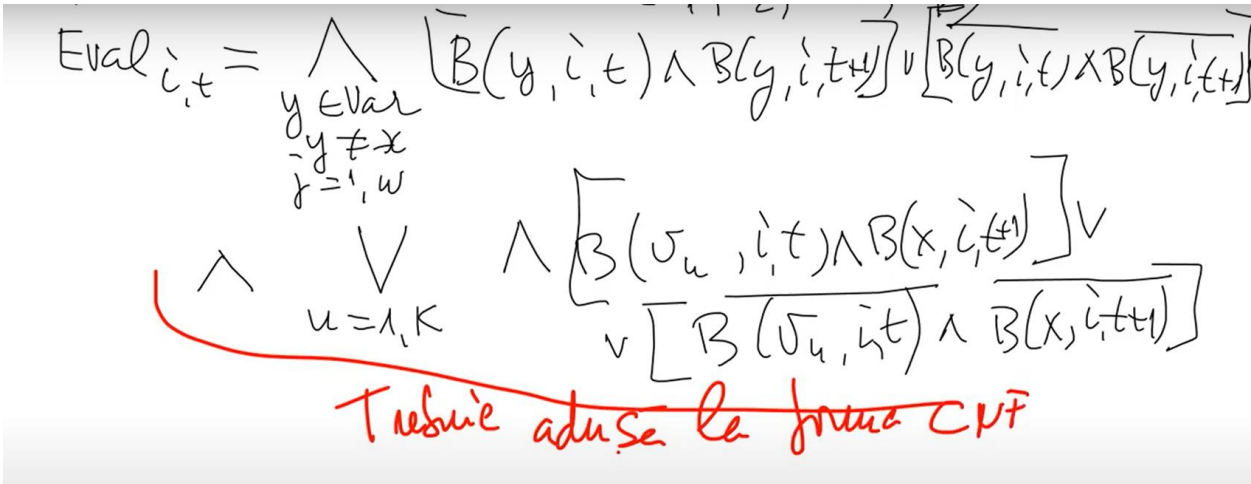
Eval $_{i, t}$ = ^ B(x, i, t) ^ B(x, i, t+1) ^ ![B(x, i, t) ^ B(x, i, t+1)]



→instanta a problemei din inteligenta artificiala – axiomele cadrului („frame axiom”)

Caz 2) Choice x←choice(v1, v2, ... , vk)

Eval $_{i, t}$ = [B(y, i, t) B(y, i, t+1)] V [!B(y, i, t) !B(y, i, t+1)] V [B(v, i, t) B(x, i, t+1)] V [!B(v, i, t) !B(x, i, t+1)]



Caz 3) Atribuire x←expresie

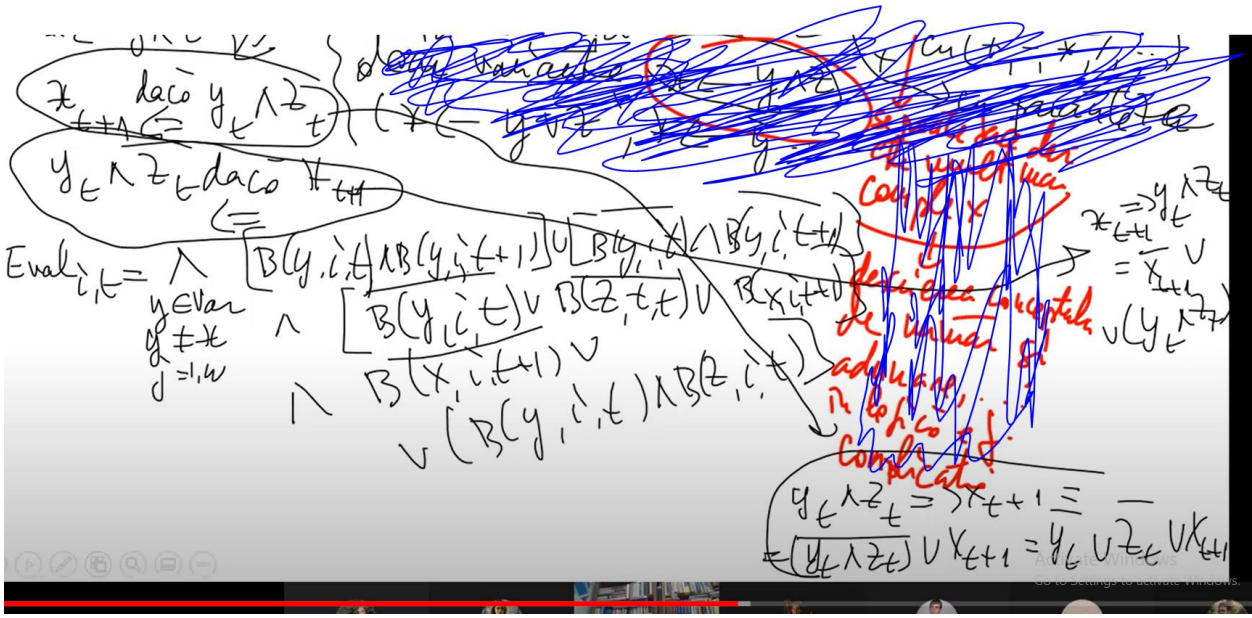
Expresie: logica, aritmetica, cu paranteze

Consideram aici doar varianta x←y z

X_{t+1} daca y_t z_t

y_t z_t daca X_{t+1}

Eval $_{i, t}$ = [B(y, i, t) B(y, i, t+1)] [!B(y, i, t) !B(y, i, t+1)] [!B(y, i, t) !B(z, i, t) B(x, i, t+1)] [!B(x, i, t+1) B(y, i, t) B(z, i, t)] – trebuie adus la CNF



6) **Stop** = V S(i, p(n)), i – starile de succes, p(n) – momentul terminarii algoritmului

Verificarea corectitudinii algoritmilor

Verificarea corectitudinii **partiale** = un algoritm Alg. este partial corect ⇔ p(i) adev (preconditie) si Alg. se termina => q(i, Alg(i)) – adev.

Verificarea corectitudinii **totale** = un algoritm Alg. este total corect daca p(i) adev (preconditie) => Alg. se termina si q(i, Alg(i)) – adev.

OBS!! In general verif corectitudinii totale este imposibila, deoarece necesita rezolvarea problemei opririi, Dar in particular, un program, se poate verifica

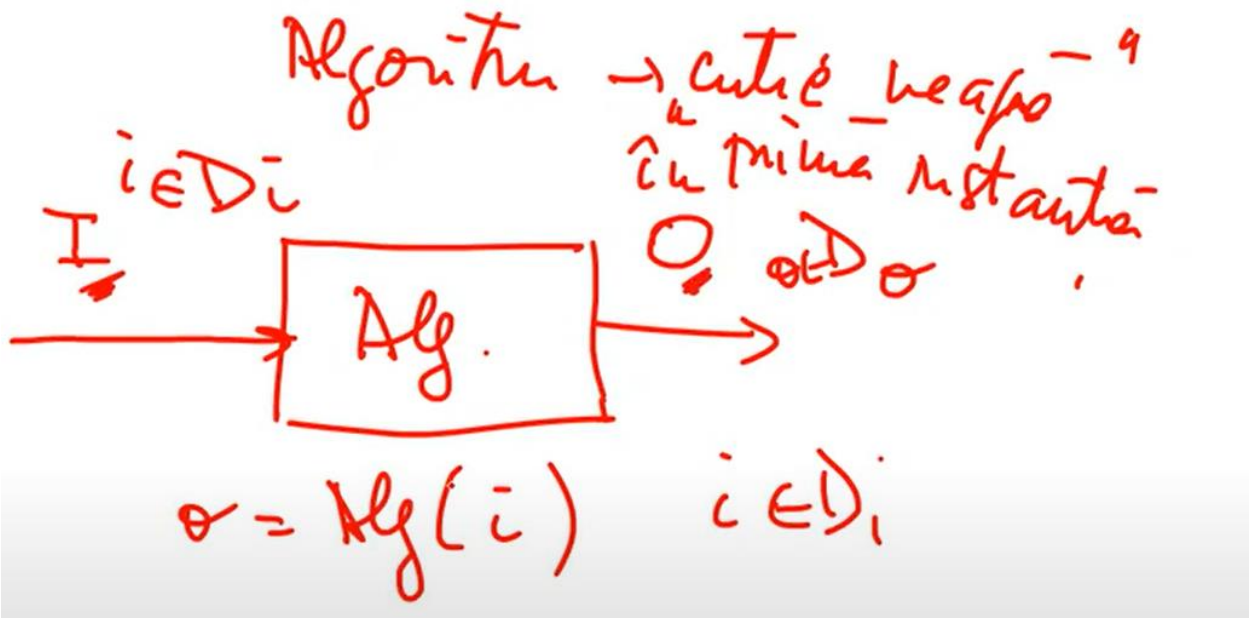
Metode verificare a corectitudinii partiale.

1) Inductie matematica

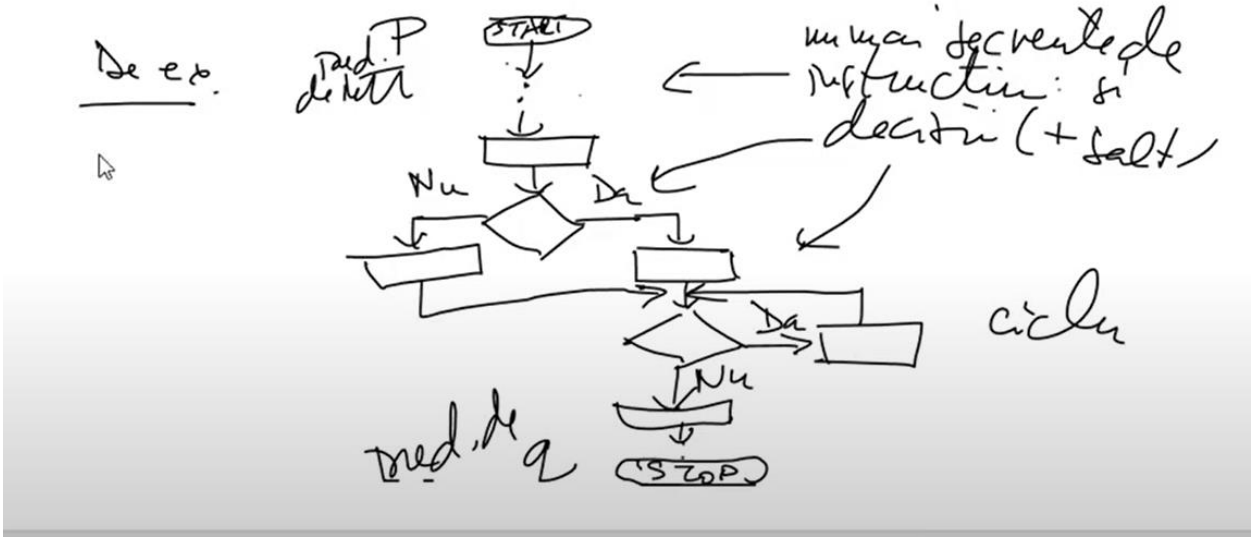
2) Metoda asertiunilor inductive - Invariantilor la ciclare (oarecum tot un fel de inductie, dar nu pe N ci pe un interval limitat pentru cicluri);

3) Metoda inductiei structurala

Metoda 2: asertiunilor inductive -> // prin 'cutie neagra' se refera la algoritm

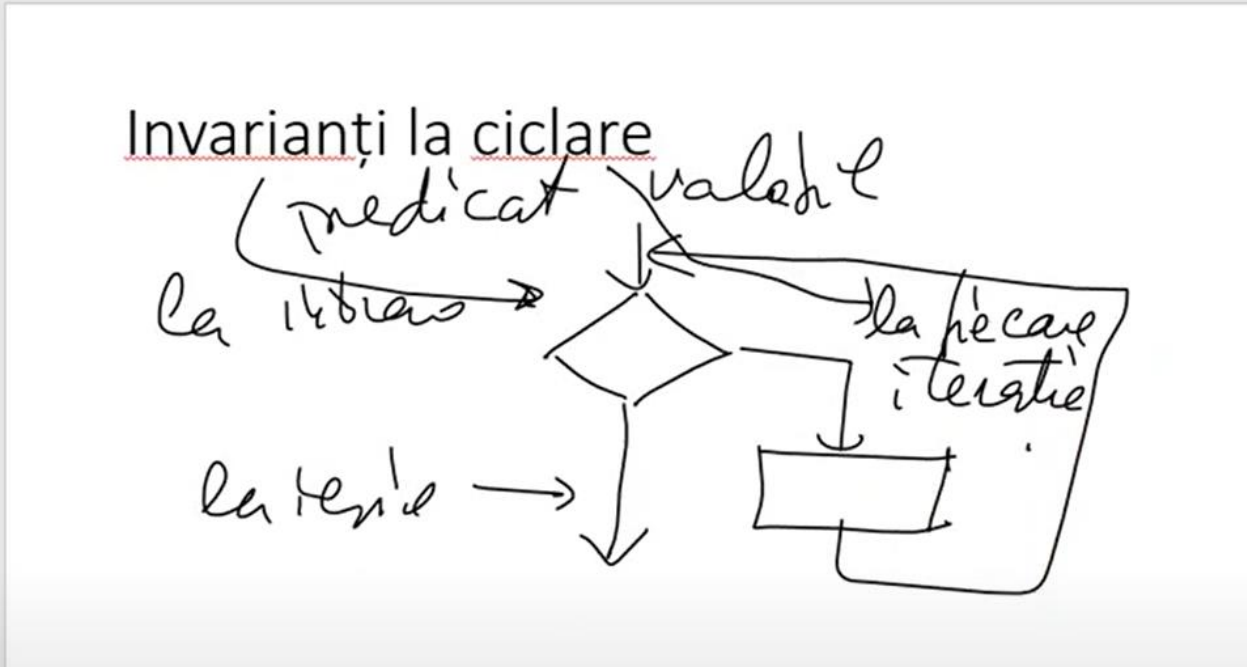


"Cutia Neagra" este accesata sub forma unei scheme logice // ca la PL



Se pun predicate inainte si dupa fiecare instrutiune si se aplica ideea de cutie neagra la fiecare instructiune

La ciclui, in plus faza de ce mai sus, trb determinat un invariant la ciclare, -> trb gasit un predicat care e valabil in cadrul ciclului, la intrare, iterare si iesire



Curs 16-17-18

Verificarea corectitudinii algoritmilor

Iesirea = Alg(intrare)
p(i) predicat de intrare, preconditie
q(i) predicat de iesire, postconditie

partial corect : p(i) si Alg se termina =>q(i) adevarat
total corect : p(i) =>Alg se termina si q(i) adevarat
(total corectitudinea nu se poate verifica in general (problema opririi), dar putemverifica daca stim programul)

Metode de verificare a corectitudinii partiale

1. Inductie matematica
2. Metoda asertiunilor inductive
   - invariatii la ciclare (oarecum tot un fel de inductie, dar nu pe N ci pe un interval limitat (pentru cicluri)
3. Inductie bine formata
4. Inductie structurala

Verificarea corectitudinii algoritmilor prin inductie

- Cea mai simpla

Exemplu : algoritmul de inserare a unui nou element intr-un heap

Se muta “varful” heapului la sfarsit, se pune noul element in varf si se propaga in jos -> se restabileste structura de heap

P(n) - demonstram caz de baza P(1)
- inductie P(n) -> P(n+1)

Daca e un heap cu n elemente, la inserare ramane heap

Verificarea corectitudinii prin asertiuni inductive

- Se considera algoritmul ca o schema logica, format din :
  - secvente de instructiuni
  - decizii
  - cicluri

- Obs. Aceasta metoda a fost introdusa inainte de limbajele cu apeluri recursive

Pentru fiecare instructiune se calculeaza cum se propaga predicatul de intrare in cel de iesire – se iau toate cazurile posibile

Exemplu:

- instructiuni de atribuire, de decizie

- la o ciclare – invariatii de ciclare – trebuie calculat invariant al ciclarii – trebuie calculate un invariant valabil la fiecare iteratie

Exemplu 1: Calculul factorialului

Algoritm Fact(n)

Daca n = 1 atunci intoarce 1

Altfel intoarce n*fact(n-1)

Date de intrare: numere intregi Z
Date de iesire: numere intregi Z
Predicatul de intrare: i apartine Z
Predicatul de iesire: i!

In acest caz algoritmul este partial corect. (Un algoritm este partial corect daca se respecta predicatul de intrare si algoritmul se termina)

Doar daca algoritmul se termina, intoarce un rezultat corect.

Doar daca P(i) = n > 1 algoritmul este total corect.

Exemplu 2 : Sistemul M I U

Construim siruri de M, I, U
Axioma (sir initial): M I
Reguli:
1. La orice sir care se termina cu I se poate adauga U
2. La M x se poate adauga un x (unde x e sir)
3. I I I se poate inlocui cu U
4. U U poate fi sters

Date de intrare: Mi
Date de iesire: sirurile cu M, I si U (potential toate combinatiile)
Predicatul de intrare:
Predicatul de iesire:

Este M U teorema? NU!

Cum demonstram?

Ce invariant putem identifica? Din el putem deduce teorema

Idee : trecere catre numere – idee generala in informatica

M : 3
I : 1
U : 0
Orice sir se transforma intr-un numar.

Ce putem spune de numerele obtinute prin aplicarea regulilor?

Obs!

1. M apare doar o data, la inceputul sirului (inca un invariant)
2. Important este numarul de I-uri ! Numerele nu sunt multiple de 3 (invariant)

- MU nu poate fi obtinut

Cum putem demonstra corectitudinea invariantilor?

a. Demonstram P(MI) – respecta cele doua proprietati (1, 2)
b. Demonstram cazul de baza

Obs!

- Descrierea sistemului M I U este in formalismul algoritmilor Markov sau a sistemelor Post – sunt echivalente cu masina Turing

- In anii 60 -70 – ideea de demonstrare – se deduceau automat dintr-un program secventele de predicate si apoi se verifica daca ultimul predicat implica predicatul de iesire – aceasta verificare era o formula logica, care trebuia sa fie demonstrata – practic trebuia demonstrata o teorema – problema – semidecidabila, de complexitate computationala mare

- gasirea invariantilor ciclurilor, in general nu este banala

Inductia bine formata

- generalizarea inductiei matematice (care ulterior este particularizata la inductia structurala, pe tda-uri)

- verificarea unei proprietati P(n) pe o multime (pentru numere naturale)

(N, <) – esential pentru inductia matematica, o multime total ordonata, < este o ordine totala pe N

Ce generalizare am putea face de la (N, <) ?

1) In loc de multime total ordonata –> partial ordonata
2) Nu trebuie sa mergem la -inf! – trebuie sa avem un cel mai mic element

Multimi bine ordonate

O multime ordonata partial care nu are siruri infinit descrescatoare, ex : (Z, <) nu este

Ex: (multimea arborilor binari, subarbori) – multime bine ordonata

({a,b,c}* (adica monoid liber generat), ”lungime mai mica”) este,(tda (tip de date abstract), sub structura) este

Teorema inductiei bine formata:

Fie (A, <) o multime bine ordonata

Oricare a apartine A, (oricare b apartine A, a < b) si din P(a) -> P(b), atunci oricare a apartine A P(A)

Exemplu de tda Arbore, ex: vid, nod(vid, 5, vid)

Tda Arbore – multimea arborilor

Sorturi: Arb, Int – numere intregi

Signatura (constructorii tda) :

vid -> Arb – operator nular
Nod: Arb x Int x Arb -> rb – operator nenular

Obs! In limbajele de programare aveti “nil” sau “null”

Tipuri de inductie:

Inductie : notatie bidimensionala (obs C.S. Perice a introdus ideea)

Spre deosebire de notatia clasica din logica predicatelor -> dezavantaj al non localizarii unor legaturi

- matematica:

P(n0), daca din P(n) -> P(n+1) rezulta oricare n>n0 P(n)

- complete:

P(n0) oricare m > n P(n) -> P(m) rezulta oricare n P(n)

- bine formata:

Oricare a din A, oricare b apartine A, daca din P(A) -> P(B) cu oricare a < b rezulta oricare a din A P(A)

Verificarea algoritmilor prin metoda inductiei structurale

Particularizare a inductiei bine formate pentru tipuri de date abstracte

Ex: tda arbori

Inductia structurala – pentru Tip, un tda, proprietate P

Pasul initial:

Oricare ar fi sigma, constructor nular, P(sigma)
Oricare ar fi sigma, constructor nerecursiv
Oricare xi apartine A, P(sigma nerecursiv(x1, x2, .. xk))

Pasul de inductie:

Oricare ar fi sigma, constructor recursiv, P(y) atunci avem P(x1, x2, .., y, xk)

Concluzia:

- > Oricare x apartine Tip, P(x)

Demonstram corectitudinea algoritmilor:

- inductie matematica – cand avem o singura directie de crestere a unei structuri de date

- inductie structurala – avem mai multe cai de modificare a structurii

Ex: algoritmul de oglindire a unui arbore binar

Mirror(nod(a1,k, a2)) -> nod (mirror(a2, k, mirror(a1))
mirror(vid) -> vid

P(a): mirror(mirror(a)) = a

Algoritm : mirror(a)

```
Daca a = vid atunci intoarce vid
        Altfel (adica a = nod(a1,k,a2))
        intoarce nod(mirror(a2), k, mirror(a1))
```

Demonstratie

a) caz de baza a = vid -> intoarce vid -> mirror(mirror(vid)) = mirror(vid) = vid

b) caz de recursivitate a = nod(a1, k, a2)

mirror(mirror(nod(a1, k, a2)) = mirror(nod(mirror(a2), k, mirror(a1)) = nod(mirror(mirror(a1), k, mirror(mirror(a2))) = nod(a1, k, a2)

dar conform ipotezei de inductie avem mirror(mirror(a1)) = a1 si lafel a2

O ierarhie spatiu-timp a problemelor

Criterii de clasificare

- spatiu-timp

- algoritmi determinisi-nedeterministi

Cazul algoritmilor determinisi

Fie Q: I -> {0,1} o problema de decizie cu date de intrare din I rezolvabila prin multimea algoritmilor determinisi AlgQ si f:N->R+

AlgQ – algoritmi determinisi care rezolva problemele Q

AlgQ T(f) = Alg apartine AlgQ oricare i din I, exista timp(Alg, Q) = O(f(n))

AlgQ S (f) = Alg apartine AlgQ oricare i din I, exista spatiu(Alg, Q) = O(f(n))

TIME(f) = Q cu AlgQT (f) != multimea vida -> toate problemele care pot fi rezolvate in O(f(n)) (complexitate temporala) cu un algoritm determinist

SPACE(f) = Q cu AlgQS (f) != multimea vida -> cu complexitaate spatiala O(f(n))

Clase de spatiu si timp

P=PTIME= Reuniunea TIME(f) unde f = polinom(n)

PSPACE = Reuniunea SPACE(f) unde f = polinom(n)

LOGTIME = Reuniunea TIME(lambda(n)(log(n)))

LOGSPACE = Reuniunea SPACE(lambda(n)(log(n)))

EXPTIME = timp exponential

EXPSPACE = spatiu exponential

Problema accesibilitatii intr-un graf (GAP)

Idee – Executia unui algoritm este vazuta din punctul de vedere al starilor prin care trece algoritmul – Spatiu al starilor

– graf cu noduri pentru stari

- arce pentru tranzitii intre stari

- exista o stare initiala

- exista o stare finala

- GAP: “Exista o cale de la starea initiala la starea finala?”

GAP -> TIME – care este complexitatea temporala a lui GAP?

Daca avem descrierea unui algoritm (determinist) prin spatial starilor, atunci complexitatea temporala a algoritmului poate fi evaluate prin complexitatea temporala a lui GAP (+ complexitatea construirii spatiului starilor)

Algoritm GAP-T(n) este: // spatial starilor este (S, A, s0, sf)

Pentru fiecare si din S repeta vizitat(si) <- fals

Vizitat(s0) <- adevarat

Parcurgere recursive(s0)

Intoarce vizitat(sf) // daca este o cale de la s0 la sf, atunci sf va fi vizitat

Algoritm parcurgere recursiv(si) este: // consideram ca avem o reprezentare a grafului prin liste de adiacente

Pentru fiecare surm = adiacenta(si)

Repeta

Daca vizitat (surm) = fals

Vizitat (surm) <- adevarat

Atunci parcurgere recursiv (surm)

Care este complexitatea temporala?

- Ea este in functie de numarul de stari (m) si tranzitie (a), fiind O(m+a) – muchii si arce

- pentru graf complet -> O(n^2)

GAP-T are complexitatea temporala O(n^2) adica PTIME

GAP -> SPACE – care este complexitatea spatiala a lui GAP?

E mai dificil de calculate pe algoritmul GAP-T, scriem un alt algoritm, GAP-S (pentru GAP-T s-ar putea sa fie chiar o complexitatea mai mare spatiala decat GAP-S)

Nu mai facem o parcurgere in adancime ca la GAP-T ci facem divide&impera

Exista drum de la so la sf (cu o anumita lungine) se reduce la doua subprobleme de gasire a unui drum de lungime/2

Algoritm GAP-S(i, j, lg)

Daca lg = 0 atunci daca i=j atunci intoarce adevarat

Altfel intoarce fals

Daca lg = 1 atunci daca i,j apartine A

Atunci intoarce adevarat

Altfel intoarce fals

Pentru k de la 1 la n-1 repeta

Daca GAP-S(I, k,lg/2) & GAP-S(k, j, lg/2)

Atunci intoarce adevarat

Altfel intoarce fals

Analiza a lui GAP-S

- Este ineficient temporal (complexitatea lui temporala va fi mai mare decat a lui GAP-T)

- dar, putem calcula usor complexitatea spatiala a lui GAP-S

- la calculul complexitatii soatiale cosideram spatial ocupat pentru variabilele algoritmului – de exemplu, pentru n variabile – log(n), unde n este dimensiunea maxima a valorii unei variabile

- la GAP-S – 4 variabile – (I, j, lg, k) – log(n) si sunt maxim log(n) apeluri recursive – rezulta ca avem complexitate spatiala O((log n)^2)

Concluzie:

GAP-T apartine TIME(n^2) = PTIME

GAP-S apartine SPACE(log(n)^2)

Cazul algoritmilor nedeterministi

- complexitati angelice – cea mai scurta cale catre “success” sau cea mai lunga catre “fail”

N - AlgQ T(f) = Alg apartine AlgQ oricare i din I, exista timp(Alg, Q) = O(f(n))

N - AlgQ S (f) = Alg apartine AlgQ oricare i din I, exista spatiu(Alg, Q) = O(f(n))

NTIME(f) = Q | N-AlgQT(f) != multimea vida

NSPACE(f) = Q cu AlgQS (f) != multimea vida

Clase:

NPTIME(NP)

NPSPACE

NLOGTIME, NLOGSPACE

Problema N-GAP pentru cazul algoritmilor nedeterministi NTIME, NSPACE

- este suficient un singur algoritm

Algoritm N-GAP (S, A, s0, sf)

```
Starea curenta <- s0; lg <- 0
Repeta
    V <- choice(S)
    Daca (stare curenta, v) nu apartine  A sau lg >= n atunci fail
    Stare curenta <- v
Pana cand starea curenta = sf
```

Cand ies din ciclu = success

Analiza N-GAP

Complexitatea temporala: N-GAP apartine NPTIME

Complexitatea spatiala: N-GAP apartine NLOGSPACE

Relatii intre clase de probleme:

PTIME este si NPTIME

NSPACE este si NPSPACE

LOGSPACE este si NLOGSPACE

LOGTIME este si NLOGTIME

Observatii – relatii intre clase de timp sau intre clase de spatiu

O ierarhie spatiu-timp a problemelor - Cursul 19

Criterii de clasificare

- Algoritmi determinisți – nedeterminisți
- Spatiu – Timp

1) Cazul algoritmilor determinisți

Fie Q:I -> {0,1} o problema de decizie cu date de intrare din I, rezolvabila prin multimea algoritmilor determinisți Alg_Q si f:N -> R+

Alg_Q – algoritmii determinisți care rezolva problemele Q

Problema accesibilitatii intr-un graf (GAP)

- Idee – Executia unui algoritm este vazuta din punctul de vedere al starilor prin care trece algoritmul – SPATIU AL STARILOR – graf cu:
- Noduri pentru stari
- Arce pentru trazitii intre stari
- Exista o stare initiala
- Exista o stare finala

Concluzie:

- GAP-T $\in$ TIME (n^2) = PTIME
- GAP-S $\in$ SPACE((lgn)^2)

Lema 1

Fie f:N->R+ calculabila cu o complexitate spatiala O(f(n)) care satisface proprietatea ca f(n) >= log(n) pentru n >= 1.

Pentru oricare algoritm Alg de decizie nedeterminist, cu spatiu f-limitat, exista k, astfel incat pentru orice instanta i a intrarii I, spatiul de stare al lui Alg(i) poate fi construit cu o complexitate temporala in O(k^f(n)), unde n este dimensiunea intrarii.

Demonstratie:

Dam un algoritm de construire al spatiului starilor – si ii calculam complexitatea temporala, stiind complexitatea spatiala.

Algoritmul are n instructiuni. Construim toate starile posibile si toate tranzitiile posibile.

Algoritmul are adrese de memorie [0, f(n)].

Algoritmul are pointeri de instructiuni [0, m].

Algoritmul are memorie (siruri de biti) [0, 2^f(n)].

Algoritm – construieste spatiul starilor (Alg, instr).

Construim starile si tranzitiile.

N <- **Ø ;** A <- ***Ø //*** stari si tranzitii

Pentru instructiuni de la 0 la n, repeta

Pentru adrese de memorie de la 0 la lungimea memoriei, repeta

Pentru pointeri de la 0 la m, repeta

Pentru memorie de la 0, la max m, repeta

N <- N U (instr, admem, ptr, mem)

Pentru fiecare stare 1 din N repeta

Pentru fiecare stare 2 din N repeta

A <- A U {(stare 1, stare 2)}

S0 -< (0,0,0,0); Sf <- (n1, n2, n3, n4)

Care este complexitatea unui algoritm?

Pentru construirea N, cele 4 cicluri

(n + 1)[f(n) + 1](n + 1)2^f(n)

2^f(n) inseamna toate sirurile de biti din memorie

Deoarece f(n) >= log n => complexitatea temporala

Construirea lui A, daca N are O(2^f(n))

2 cicluri dupa N => O(2^2f(n)) = O(K^f(n))

Din spatiu O(f(n)) => construirea spatiul starilor in O(K^f(n))

Teorema

- Pentru oricare Problema Q, Q ∈ NSPACE(f) -> Q ∈ TIME(c^f)
- Demonstratie – facem o diagrama comutativa

Alg ----------> (N, A, s0, sf)

Alg – nedeterminist si rezolva Q => conform lemei,.

Complexitatea contr. Sps este O(K^f(n)) temporal

Complex. GAP-T este O(n^2) => total O((K ^ f(n)) ^ 2) = O(c ^ f(n))

=> Q ∈ NSPACE(f) => Q ∈ TIME(c^f)

Corolar

- NLOGSPACE $\subseteq$ PTIME
- Demonstratie

Din teorema avem Q $\in$ NSPACE(f) -> Q $\in$ TIME (c ^ f)

Fie f=log(n) => NLOGSPACE => TIME(c^log(n)) = PTIME => NLOGSPACE $\subseteq$ PTIME

Lema 2 (Obs.: Este foarte asemanatoare cu Lema 1, diferenta este ca ne intereseaza complexitatea spatiala)

Fie f:N->R+ calculabila cu o complexitate spatiala O(f(ncare satisface proprietatea ca f(n) >= log(n) pentru n >= 1.

Oricare algoritm Alg de decizie dnedeterminist, cu spatiu f-limitat, astfel incat pentru orice instanta i a intrarii I, spatiul de stare al lui Alg(i) poate fi construit cu o complexitate spatiala in O(f(n)), unde n este dimensiunea intrarii.

Demonstratie:

- Acum construim spatiul de stare, dar nu ne intereseaza complexitatea temporala, ci cea spatiala => ne intereseaza un minim de spatiu ocupat -> rescriem algoritmul anterior contruieste sps eliminand ce nu este obligatoriu care ocupa mult spatiu -> eliminam din memoria interna N si A -> le tinem undeva global (extern).

Teorema

- Pentru oricare problema Q, Q $\in$ NSPACE(f) -> Q $\in$ SPACE (f ^ 2)

Corolar

- PSPACE = NPSPACE
- Demonstratie

1) PSPACE ⊆ NPSPACE – una din incluziunile evidente

Din Teorema, NSPACE(f) ⊆ SPACE (f ^ 2) => f = polinom

2) NSPACE ⊆ PSPACE

Din 1 si 2 => PSPACE = NPSPACE

Algoritmi de aproximare

- Ratiunea de a fi: Necesitatea de a rezolva probleme pentru care exista doar algoritmi NP-complete sau NP-dificili
- Intrebare esentiala: Cat de aproximativa e solutia?

E necesara o masura

- Intrebare suplimentara: Algoritmul de aproximare este polinomial sau nu?

  Nu orice algoritm e bun

Sunt probleme la care nici nu exista un algoritm de aproximare polinomial cu un factor de aproximare dat

Exemplu – Problema alocarii de resurse

- Repartizarea optima a n procese de m procesoare. Fiecare proces dureaza un timp cunoscut apriori.

P1, P2, … , Pn procese

Q1, q2, …, Qn procesoare

D1, d2, …, dn

D - total sa fie minim (problema de optim, minimizare)

- Euristica – solutie care rezolva problema de multe ori aproximativ

1) alocate aleatoriu – cel mai defavorabil D este de 2 ori cea optima

2) sortam procesele descrescator dupa durata si le alocam in aceasta ordine – 4/3 ori in cazul cel mai defavorabil

Factor de aproximare

Delta (n) = max {i $\in$ I | dim(i) = n * cost(i) / cost * (i)}

Pentru minimizare

Cost(i) - costul aproximativ

Cost * (i) - costul optim

Costul aproximativ este mai mare decat costul optim.

Delta (n) = max {i $\in$ I | dim(i) = n cost * (i) / cost(i)}

Eroarea relativa de aproximare

$$\varepsilon(n) = \max\{_{i \in} \text{ I | dim(i) = n * |cost(i) - cost*(i)| / cost * (i) }$$

Exemplu 2 – arc conectivitate

Pentru un graf neorientat dat sa se gaseasca un subgraf 2 – arc conectat – daca se elimina un arc, graful ramane conex (graf biconectat).

Algoritmi de aproximare

Ratiunea de a fi : necesitatea de a rezolva probleme pt care exista doar algoritmi NP-complete sau NP-dificili (S-a discutat in capitolele anterioare de cazul simplu al problemelor de decizie, dar sunt si probleme de optimizare)

Intrebare esentiala: Cat de aproximativa este solutia? (E necesara o masura)

Intrebare suplimentara: Algoritmul de aproximare este polinomial sau nu?

-nu orice alg e bun

-sunt probleme la care nici nu exista un alg de aproximare polinomial cu un factor de aproximare dat

Exemple:

– Problema alocarii de resurse

-2-arc conectivitate

(Cursul trecut)

Exemplu: Acoperirea cu noduri a unui graf („Vertex Cover”)

Dandu-se un graf neorientat G = (N,A), sa se gaseasca o submultime M a lui N, cu numar minim de noduri, astfel incat fiecare muchie din A sa aiba cel putin un nod in M.

$M \subset N$

M = {b, c, f}

Problema de optim

Cum reformulam in problema de decizie? -> k-acoperire: Exista o acoperire cu noduri de dimensiune k?

Este in NP? OK

Algoritm de aproximare pentru acoperirea cu noduri

Algoritm acoperire nod aproximativ(N, A; M)

M<- Ø

Pentru fiecare (u, v) A repeta

Daca $u \notin M$, atunci M <- M U {u}

Daca $v \notin M$, atunci M <- M U {v}

Elimina (u, v) si arcele adiacente la u si v

Exemplu caz defavorabil extrem pt graful dat si alg de aproximare

(a, b) -> M = {a, b}

(e, f) -> M = {a, b, e, f}

(c, d) -> M = {a, b, e, f, c, d}

La fiecare pas cate 2 noduri, incluzand pe cele din multimea optima

Care este factorul de aproximare? ≤ 2

λ conectivitate intr-un graf

Fie G = (N, A) un graf conex neorientat

λ – arc conectivitate

Ramane G conex daca se elimina λ – 1 arce?

Daca λ = 2 => problema biconectarii unui graf

Exemplu – in retele -> daca „cade” o conexiune, sa existe o cale alternativa (desen)

λ – nod conectivitate

Ramane G conex daca se elimina λ – 1 noduri?

λ – arc conectivitate – algoritmi de aproximare

ideal ar fi sa gasim un subgraf optim cu numar minim de arce G optim = (N, A optim), A optim inclus in A, sa fie conex. G optim este obtinut din G prin leiminarea a λ – 1 arce.

#A optim = #A – (λ – 1)

Khuller-Viskin

Mai multe treceri prin graf

Nagamochi – Ibaraki

Necesita o singura trecere prin graf

!Urmeaza desen!

G = (N, A)

N = {a, b, ... , i}

Optim ar fi ciclul hamiltonian

Nr arce = 9

Pt λ = 2 => a, b, i, h, g, e, f, d, c, a

Algoritmul Khuller-Viskin

2 etape:

Se construieste un graf G2 care sa fie 2-arc conectat

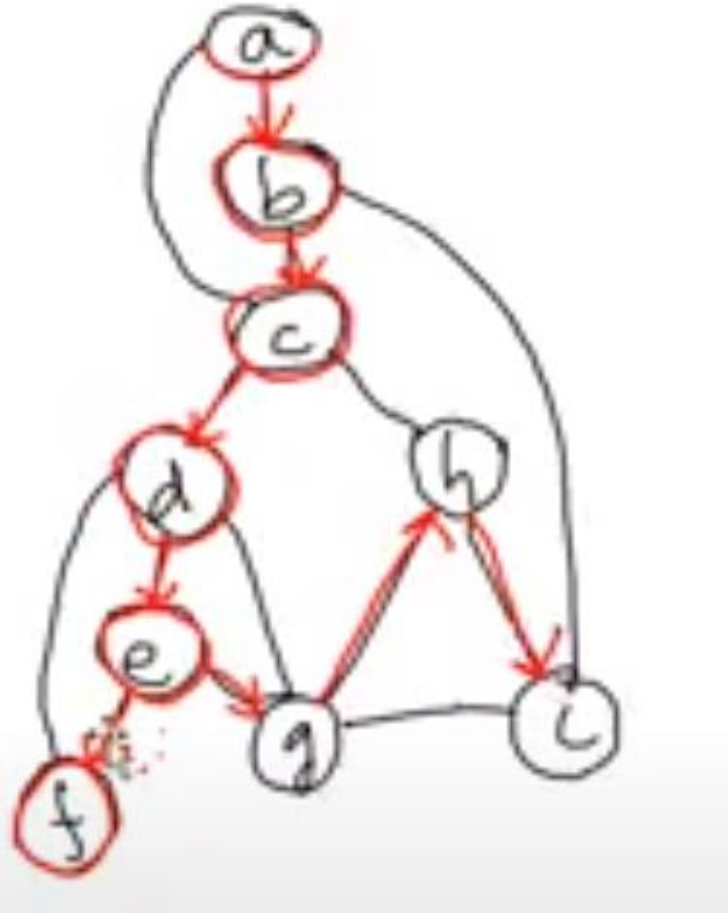
Se face o parcurgere in adancime si se adauga arce la arborele de parcurgere astfel incat sa fie 2 – arc conectat

Se face un ciclu de la 2 la λ in care se construiesc grafuri 3-arc conectate, 4-arc conectate, ... λ-arc conectate, adaugand noi arce

Etapa 1 – parcurgere in adancime – construieste un arbore de parcurgere in adancime

!Desen!

Exista mai multe variante de parcurgere



Combinam parcurgerea in adancime cu adaugarea de arce pentru a avea 2 arc conectivitate (algoritm Giumale, extindere a algoritmului de detectare a puntilor – cand e detectat o punte, se adauga un arc, astfel incat sa nu mai fie punte).

Teorema

G2 = (N, A2) graful generat la pasul 1 din Alg. Khuller –Viskin este 2-arc conectat

Demonstratie – prin constructie s-au adaugat arce pt eliminarea puntilor

G2 este o aproximare a solutiei optime.

Arbore de partitionare

Fie G = (N, A) un graf neorientat, atunci un arbore de partitionare al lui G cu dimensiunea K este T_G = (N_G, A_G), unde N_G = {N1, N2, …, Nk} cu proprietatea ca:

1) N = N1 U N2 U … U Nk

N -> o multime de noduri din N

2) Oricare ar fi i, j = 1, …, K Ni intersectat cu Nj = multimea vida

3) Oricare ar fi u ∈ Ni => Oricare ar fi v ∈ N avem (U, v) ∈ A

Avem ori v ∈ Ni ori v ∈ tata(Ni) ori v ∈ fiul (Ni)

Teorema

Fie G = (N, A) un graf 2-arc conectat si G2 = (N, A2) un subgraf construit din algoritmul de la etapa 1din Khuller – Viskin

Fie A2 (cu bara jos, gen subliniat) arcele rezultate din A din care s au scos arcele de partitionare (in ex {alfa, beta, gamma}) si cele adaugate

Se poate obtine astfel un arbore de partitionare al lui G

Teorema #(A2) >= 2(K-1)

Fie G = (N, A) 2-arc conectat si G2 = (N, A2) construit in etapa 1 Khuller – Viskin si T_G un arbore de partitionare a lui G de dim K

Atunci numarul de arce din A2, adica #A(2) >= 2(K-1)

Dem. T_G are K noduri => are K-1 arce

La algoritmul etapa 1, se adauga k-1 arce la cele k-1 de partitionare => #(A2) >= 2(K-1)

Corolar: Numarul optim de arce Opt(G) >= max[n, 2(k-1)]

Dem. - optim este ori ciclu hamiltonian (adica n) ori >= 2(k-1) din teorema anterioara

Teorema: Delta < 3 / 2 = 2 – 1 / Lambda ; Lambda = 2

Dem. Delta(n) = #A2 / Opt(G) <= (n-1 + k-1) / max(n, 2k-1)

N-1 -> numarul de arce din arborele de parcurgere in adancime

K-1 -> numarul de arce adaugate

Luam cele 2 posibiltati

Max = n => n > 2(k-1) => n/2 +1 > k => k <= n/2

Delta(n) <= (n-1 + n/2) / n = 3 / 2

si max = 2k – 1

2k-1 > n

Delta(n) <= (2k-1-1+k-1)/(2k-1) < 3/2

====> Delta < 3/2

Algoritmul Khuller – Viskin

2 etape:

1. Se construieste un graf G2 2-arc conectat

   Se face o parcurgere in adancime si se adauga arce la arborele de parcurgere astfel incat sa fie 2-arc conectat

2. Se face un ciclu de la 2 la Lambda in care se construiesc grafuri 3-arc conectate, 4-arc conectate, … Lambda-arc conectate, adaugand noi arce

G2 -> G3 -> G4 -> … -> G_lambda

Duritatea algoritmilor de aproximare

- Exista probleme pentru care un algoritm polinomial de aproximare cu un anumit factor poate exista doar daca P=NP
- Fie Q o problema pentru care exista un algoritm polinomial de aproximare cu Delta(n) <= f(n) doar daca P=NP. Spunem ca f-aproximarea problemei Q este dura sau, alternativ, ca facotrul de aproximare Delta(n) = f(n) este dur

- Exemplu de problema cu factor de aproximare dur – problema colorarii unui graf: Se poate colora un graf neorientat folosind doar q-culori?
- Numarul cromatic al unui graf = numarul minim de culori cu care se pot colora nodurile unui graf astfel incat doua noduri adiacente sa aiba culori diferite

Colorarea unui graf -> problema de decizie . (are G numarul cromatic K?)

-> problema de optim(care e numarul cromatic minim al lui G?}

1 Problema accesibilitatii intr-un graf(GAP)

- Executia unui algoritm este vazuta din punct de vedere al starilor prin care trece algoritmul – Spatiu al starilor
- Nodurile in graf reprezinta stari
- Arcele reprezinta tranzitii intre doua stari
- Exista o stare initiala si una finala
- Problema GAP verifica daca exista o cale de la starea initiala la starea finala, varf sursa-varf destinatie
- Complexitatea temporala(GAPTime) si cea spatiala(GAPSpace) al unui algoritm determinist:

$$GAPTime \; \epsilon \; Time(n^2) = PTime(timp \; polinomial)$$

$$GAPSpace \; \epsilon \; Space((logn)^2)$$

- Complexitatea temporala(NGAPTime) si cea spatiala(NGAPSpace) al unui algoritm nedeterminist:

$$NGAPTime \; \epsilon \; NPTIME(nedeterminist \; polinomial)$$

$$NGAPSpace \; \epsilon \; NLOGSPACE$$

-Pentru un algoritm de decizie nedeterminist:
* Pentru o complexitate spatiala O(f(n)), unde f(n) $\geq log(n), n \geq 1, complexitatea \; temporala \; este$: $O((k)^{f(n)})$

2 Algoritmi de aproximare

- Rezolva probleme pentru care exista algoritmi NP-complete sau NP-dificili cu un factor de aproximare dat - Factori de aproximare pentru n, n fiind intrare:

$$Factor \; de \; aroximare \; pentru \; minimizare : \delta(n) = max \; \{i \epsilon I | dim(i) = n \frac{cost(i)}{cost^*(i)}\}$$

$$Factor \; de \; aroximare \; pentru \; maximizare : \delta(n) = max \; \{i \epsilon I | dim(i) = n \frac{cost^*(i)}{cost(i)}\}$$

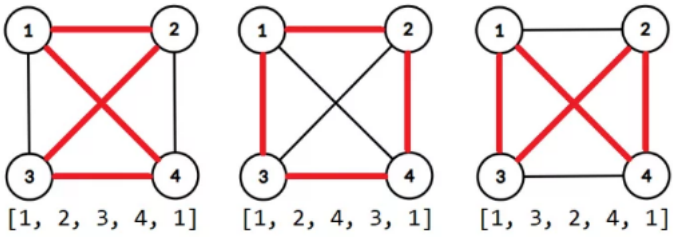
unde cost(i) - cost aproximativ, cost*(i)- cost optim

- Erorea relatixa de aproximare

$$\varepsilon(n) = max\{i \epsilon I, dim(i) = n \frac{|cost(i) - cost^*(i)|}{cost^*(i)}\}$$

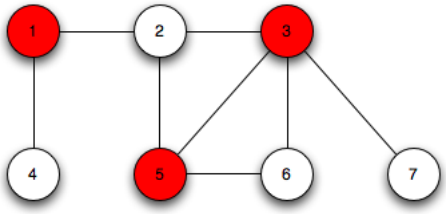
2.1 2-arc conectivitate

- Pentru grafuri NEORIENTATE se gaseste un subgraf 2-arc conectat, daca se elimina un arc din acest subrgaf, ramane conex - Solutia optima este sa folosim ciclul hamiltonian - trece prin toate nodurile o singura data si foloseste un numar minim de arce.



2.2 Acoperirea cu noduri a unui graf. Vertex Cover

- Pentru un graf NEORIENTAT G = (N,A), sa se gaseasca o submultime M a lui N, cu un numar minim de noduri, astfel in cat fiecare muchie din A, sa aiba cel putin un nod in M.
- Factorul de aproximare este $\delta \leq 2$



2.3 $\lambda - arc\ conectivitate\ intr - un\ graf$

(Se elimina $\lambda - 1$ *arce*)

- Se doreste gasirea unui subgraf cu un numar minim de arce si sa contina toate nodurile.

- Algoritmul Khuller-Viskin: Factorul de aproximare: $\delta < 2 - \frac{1}{\lambda}$

- Cum functioneaza algoritmul Khuller-Viskin:
  * Se parcurge graful in adincime si in paralel se construieste un arbore de parcurgere in adancime. Se adauga arce la arborele de parcurgere incat sa fie 2-arc conectat.
  * Se face un ciclu de la 2 la λ.

* Se construiesc grafuri 3-arc conectate, 4-arc conecate pana la $\lambda - arc\ conectate$.

- Algoritmul Nagamochi-Ibaraki: Factorul de aproximare $\delta = 2$

2.4 $\lambda - nod\ conectivitate\ intr - un\ graf$

(Se elimina $\lambda - 1$ *noduri*)

2.5 Arbore de partitionare

- Pentru G = (N, A) - graf neorientat, arborele de partitionare al lui G de dimensiunea K este

$$T_G = (N_G, A_G), unde\ N_G = \{N_1, N_2 ... N_K\}$$

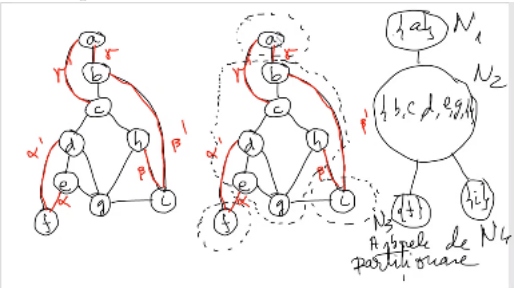
- Cu proprietatile:

$$1) N_k\ este\ o\ multime\ de\ noduri\ din\ N: \ N = N_1 \cup N_2 \cup ... N_k$$

$$2) Nodurile\ sunt\ dijuncte\ \forall i, j = 1, ...k, N_i \cap N_j = \emptyset$$

$$3) \forall u \epsilon N_i => \forall v \epsilon N\ unde\ u\ si\ v\ formeaza\ un\ arc => v \epsilon N_i, sau\ tatalui/fiului\ lui\ N_i$$

- Teorema: Fie G = (N,A) un graf 2-arc conectat si G2 = (N1, A2) un subgraf costruit cu Kuller Viskin. Fie A2' cu arcele rezultate din A din care s-au scos arcele de partitionare ori cele adaugate. Astfel se poate obtine un arbore de partitionare al lui G.



$\alpha,\ \beta,\ \gamma - sunt\ arcele, k = 4$

- Numarul optim de arce Opt(G) $\geq max[N, 2(K-1)]$

$-\ Delta\ in\ acest\ caz\ \delta < \frac{3}{2}$

2.6 Duritatea algoritmilor de aproximare

- Exista probleme pentru care un algoritm polinomial de aproximare cu un anumit factor poate exista doar daca P = NP

- Fie Q o problema pentru care exista un algoritm polinomial de aproximare cu $\delta(n) \leq f(n)$ *doar daca* $P = NP$ *atunci* :

f-aproximarea problemei Q este dura, sau factorul de aproximare $\delta(n) = f(n)$ *este dur.*